Chapter 11
Operator Overloading, Copy/Move Semantics and Smart Pointers

Learning Objectives

- Use built-in string class overloaded operators.
- Use operator overloading to help you craft valuable classes.
- Understand when to implement the special member functions for custom types.
- Understand when objects should be moved vs. copied.
- Use rvalue references and move semantics to eliminate unnecessary copying when objects go out of scope, improving performance.
- Manage dynamic memory automatically with smart pointers.
- Craft a MyArray class that defines the five special member functions to support copy and move semantics, and overloads many unary and binary operators.
- Use C++20's three-way comparison operator (<=>).
- Convert objects to other types.
- Use explicit to prevent constructors and conversion operators from being used for implicit conversions.
- Experience a "light-bulb moment" when you'll truly appreciate the elegance and beauty of the class concept.

Introduction

- This chapter presents **operator overloading**, which enables C++'s existing operators to work with custom class objects.
- One example of an overloaded operator in standard C++ is <<, which we use as
  - the stream insertion operator and
  - the bitwise left-shift operator
- Similarly, we use >> as
  - the stream extraction operator and
  - the bitwise right-shift operator

Introduction (Cont.)

- You've already used many overloaded operators.
  - Some are built into the core C++ language itself.
  - For example, the + operator performs differently, based on its context in integer, floating-point and pointer arithmetic with data of fundamental types.
  - Class string also overloads + to concatenate strings.
- You can overload most operators for use with custom class objects.
  - The compiler generates the appropriate code based on the operand types.
  - The jobs performed by overloaded operators also can be performed by explicit function calls, but operator notation is often more natural and more convenient.

Introduction (Cont.)

- string Class Overloaded Operators Demonstration
  - In Chapter2's Objects-Natural case study, we introduced the standard library's string class and demonstrated a few of its features, such as string concatenation with the + operator.
  - Here, we demonstrate many other string-class overloaded operators, so you can appreciate how valuable operator overloading is in a key standard library class before implementing it in your own custom classes.
  - Next, we'll present operator-overloading fundamentals.

Introduction (Cont.)

- Dynamic Memory Management and Smart Pointers
  - We introduce dynamic memory management, which enables a program to acquire the additional memory it needs for objects at runtime rather than at compile-time and release that memory when it's no longer needed so it can be used for other purposes.
  - We discuss the potential problems with dynamically allocated memory, such as forgetting to release memory that's no longer needed—known as a **memory leak**.
  - Then, we introduce smart pointers, which can automatically release dynamically allocated memory for you.
  - As you'll see, when coupled with the **RAII (Resource Acquisition Is Initialization)** strategy, smart pointers enable you to eliminate subtle memory leak issues.

Introduction (Cont.)

- MyArray Case Study
  - Next, we present one of the capstone case studies.
  - We build a custom MyArray class that uses overloaded operators and other capabilities to solve various problems with C++'s native pointer-based arrays.
  - We introduce and implement the five special member functions you can define in each class—**the copy constructor, copy assignment operator, move constructor, move assignment operator and destructor**.
  - Sometimes the default constructor also is considered a special member function.
  - We introduce copy semantics and move semantics, which help tell a compiler when it can move resources from one object to another to avoid costly, unnecessary copies.

Introduction (Cont.)

- MyArray Case Study
  - MyArray uses smart pointers and RAII, and overloads many unary and binary operators, including
    - = (assignment),
    - == (equality),
    - != (inequality),
    - [] (subscript),
    - ++ (increment),
    - += (addition assignment),
    - >> (stream extraction) and
    - << (stream insertion).
  - We also show how to define a conversion operator that converts a MyArray to a true or false bool value, indicating whether a MyArray has elements.

Introduction (Cont.)

- MyArray Case Study
  - **Working through the MyArray case study is a “light bulb moment,” helping people truly appreciate what classes and object technology are all about.**
  - Once you master this MyArray class, you’ll indeed understand **the essence of object technology**—crafting, using and reusing valuable classes, and sharing them with colleagues and perhaps the entire C++ open-source community.

Introduction.

- C++20’s Three-Way Comparison Operator (<=>)
  - We introduce C++20’s new three-way comparison operator (<=>)—known as the **“spaceship operator**.”
- Conversion Operators
  - We discuss overloaded conversion operators and conversion constructors in more depth, demonstrating how **implicit conversions can cause subtle problems**.
  - Then, we use explicit to prevent those problems.

Using the Overloaded Operators of Standard Library Class string

- Chapter 8 presented the string class. The next program demonstrates many of string's overloaded operators and other useful member functions, including **empty**, **substr** and **at**:
  - empty determines whether a string is empty,
  - substr (for "substring") returns a portion of an existing string and
  - at returns the character at a specific index in a string (after checking that the index is in range).

Using the Overloaded Operators of Standard Library Class string (Cont.)

- Creating string and string_view Objects and Displaying Them with cout and Operator <<
  - Lines 9–12 create three strings and a string_view:
    - the string s1 is initialized with the literal "happy",
    - the string s2 is initialized with the literal " birthday",
    - the string s3 uses string's default constructor to create an empty string and
    - the string_view v is initialized to reference the characters in the literal "hello".

```
1  // main.cpp
2  // Standard library string class test program.
3  #include <format>
4  #include <iostream>
5  #include <string>
6  #include <string_view>
7
8  int main() {
9     std::string s1{"happy"}; // initialize string from char*
10    std::string s2{" birthday"}; // initialize string from char*
11    std::string s3; // creates an empty string
12    std::string_view v{"hello"}; // initialize string_view from char*
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- Creating string and string_view Objects and Displaying Them with cout and Operator <<
  - Lines 15–16 output these three objects, using cout and operator <<, which string and string_view overload for output purposes.

```
14    // output strings and string_view
15    std::cout << "s1: \"" << s1 << "\"; s2: \"" << s2
16       << "\"; s3: \"" << s3 << "\"; v: \"" << v << "\"\n\n";
```

```
s1: "happy"; s2: " birthday"; s3: ""; v: "hello
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- Comparing string Objects with the Equality and Relational Operators
  - Lines 19–25 show the results of comparing s2 to s1 using string's overloaded equality and relational operators.
  - These perform lexicographical comparisons using the numerical values of the characters in each string.
  - **When you use the format function to convert a bool to a string, it produces "true" or "false".**

```
18    // test overloaded equality and relational operators
19    std::cout << "The results of comparing s2 and s1:\n"
20       << std::format("s2 == s1: {}\n", s2 == s1)
21       << std::format("s2 != s1: {}\n", s2 != s1)
22       << std::format("s2 > s1: {}\n", s2 > s1)
23       << std::format("s2 < s1: {}\n", s2 < s1)
24       << std::format("s2 >= s1: {}\n", s2 >= s1)
25       << std::format("s2 <= s1: {}\n\n", s2 <= s1);
```

```
The results of comparing
 s2 and s1:
s2 == s1: false
s2 != s1: true
s2 > s1: false
s2 < s1: true
s2 >= s1: false
s2 <= s1: true
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- string Member Function empty
  - Line 30 uses string member function empty, which
    - returns true if the string is empty;
    - otherwise, it returns false.
  - The object s3 is empty, as we initialized it with the default constructor.

```
27    // test string member function empty
28    std::cout << "Testing s3.empty():\n";
29
30    if (s3.empty()) {
31       std::cout << "s3 is empty; assigning s1 to s3;\n";
32       s3 = s1; // assign s1 to s3
33       std::cout << std::format("s3 is \"{}\"\n\n", s3);
34    }
```

```
Testing s3.empty():
s3 is empty; assigning s1 to s3;
s3 is "happy"
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- string Copy Assignment Operator
  - Line 32 demonstrates string's overloaded copy assignment operator by assigning s1 to s3.
  - Line 33 outputs s3 to demonstrate that the assignment worked correctly.

```
27    // test string member function empty
28    std::cout << "Testing s3.empty():\n";
29
30    if (s3.empty()) {
31       std::cout << "s3 is empty; assigning s1 to s3;\n";
32       s3 = s1; // assign s1 to s3
33       std::cout << std::format("s3 is \"{}\"\n\n", s3);
34    }
```

```
Testing s3.empty():
s3 is empty; assigning s1 to s3;
s3 is "happy"
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- string Concatenation and string-Object Literals
  - Line 37 demonstrates string's overloaded += operator for string concatenation assignment.
    - We append the contents of s2 to s1, modifying its value.
  - Then line 38 outputs the updated s1.
  - Line 41 demonstrates that you also may append a C-string literal to a string object using +=.
  - Line 42 displays the result.

```
36    // test overloaded string concatenation assignment operator
37    s1 += s2; // test overloaded concatenation
38    std::cout << std::format("s1 += s2 yields s1 = {}\n\n", s1);
39
40    // test string concatenation with a C string
41    s1 += " to you";
42    std::cout << std::format("s1 += \" to you\" yields s1 = {}\n\n", s1);
```

```
s1 += s2 yields s1 = happy birthday

s1 += " to you" yields s1 = happy birthday to you
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- string Concatenation and string-Object Literals
  - Similarly, line 46 concatenates s1 with **a string-object literal, indicated by placing the letter s immediately after the closing " of a string literal**, as in
    ", have a great day!"s
  - The preceding literal actually calls a C++ standard library function that returns a string object containing the literal's characters.
  - Lines 47–48 display s1's new value.

```
44    // test string concatenation with a string-object literal
45    using namespace std::string_literals;
46    s1 += ", have a great day!"s; // s after " for string-object literal
47    std::cout << std::format(
48       "s1 += \", have a great day!\"s yields\ns1 = {}\n\n", s1);
```

```
s1 += ", have a great day!"s yields
s1 = happy birthday to you, have a great day!
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- string Member Function substr
  - Class string provides member function substr (lines 53 and 58) to return a string containing a portion of the string object on which the function is called.
    - Line 53 obtains a 14-character substring of s1 starting at position 0.
    - Line 58 obtains a substring starting from position 15 of s1.
      - If you do not provide a second argument to substr, it returns the remainder of the string.

```
50    // test string member function substr
51    std::cout << std::format("{} {}\n{}\n\n",
52       "The substring of s1 starting at location 0 for",
53       "14 characters, s1.substr(0, 14), is:", s1.substr(0, 14));
54
55    // test substr "to-end-of-string" option
56    std::cout << std::format("{} {}\n{}\n\n",
57       "The substring of s1 starting at",
58       "location 15, s1.substr(15), is:", s1.substr(15));
```

```
The substring of s1 starting at location 0
for 14 characters, s1.substr(0, 14), is:
happy birthday
The substring of s1 starting at location 15,
s1.substr(15), is:
to you, have a great day!
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- string Copy Constructor
  - Line 61 creates string object s4, initializing it with a copy of s1.
    - This calls string's copy constructor, which copies the contents of s1 into the new object s4.
    - You'll see how to define a copy constructor for a custom class later.

```
60    // test copy constructor
61    std::string s4{s1};
62    std::cout << std::format("s4 = {}\n\n", s4);
```

```
s4 = happy birthday to you, have a great day!
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- Testing Self-Assignment with the string Copy Assignment Operator
  - Line 66 uses string's overloaded copy assignment (=) operator to demonstrate that it performs **self-assignment** properly, so s4 still has the same value after the self-assignment.
    - When we build class MyArray later in the chapter, we'll show how to properly implement self-assignment for objects that manage their own memory.

```
64    // test overloaded copy assignment (=) operator with self-assignment
65    std::cout << "assigning s4 to s4\n";
66    s4 = s4;
67    std::cout << std::format("s4 = {}\n\n", s4);
```

```
assigning s4 to s4
s4 = happy birthday to you, have a great day!
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- Initializing a string with a string_view
  - Line 71 demonstrates string's constructor that receives a string_view, in this case, copying the character data represented by the string_view v (line 12) into the new string s5.

```
69      // test string's string_view constructor
70      std::cout << "initializing s5 with string_view v\n";
71      std::string s5{v};
72      std::cout << std::format("s5 is {}\n\n", s5);
```

```
initializing s5 with string_view v
s5 is hello
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- string's [] Operator
  - Lines 75–76 use string's overloaded [] operator in assignments to create lvalues for replacing characters in s1.
  - Lines 77–78 output s1's new value.

```
74    // test using overloaded subscript operator to create lvalue
75    s1[0] = 'H';
76    s1[6] = 'B';
77    std::cout << std::format("{}:\n{}\n\n",
78       "after s1[0] = 'H' and s1[6] = 'B', s1 is", s1);
```

```
after s1[0] = 'H' and s1[6] = 'B', s1 is
Happy Birthday to you, have a great day!
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- string's [] Operator
  - **The [] operator returns the character at the specified location as a modifiable lvalue or a nonmodifiable lvalue** (e.g., a const reference), depending on the context in which the expression appears.

```
74      // test using overloaded subscript operator to create lvalue
75      s1[0] = 'H';
76      s1[6] = 'B';
77      std::cout << std::format("{}:\n{}\n\n",
78         "after s1[0] = 'H' and s1[6] = 'B', s1 is", s1);
```

```
after s1[0] = 'H' and s1[6] = 'B', s1 is
Happy Birthday to you, have a great day!
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- string's [] Operator
  - For example:
    - **When using [] on a non-const string, the function returns a modifiable lvalue**, which you can place on the left of an assignment (=) operator to assign a new value to that location in the string, as in lines 77–78.
    - **When using [] on a const string, the function returns a nonmodifiable lvalue** that you can use to obtain, but not modify, the value at that location in the string.

```
74    // test using overloaded subscript operator to create lvalue
75    s1[0] = 'H';
76    s1[6] = 'B';
77    std::cout << std::format("{}:\n{}\n\n",
78       "after s1[0] = 'H' and s1[6] = 'B', s1 is", s1);
```

```
after s1[0] = 'H' and s1[6] = 'B', s1 is
Happy Birthday to you, have a great day!
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- string's [] Operator
  - The overloaded [] operator does not perform bounds checking.
  - So, you must ensure that operations using this operator do not accidentally manipulate elements outside the string's bounds.

```
74      // test using overloaded subscript operator to create lvalue
75      s1[0] = 'H';
76      s1[6] = 'B';
77      std::cout << std::format("{}:\n{}\n\n",
78         "after s1[0] = 'H' and s1[6] = 'B', s1 is", s1);
```

```
after s1[0] = 'H' and s1[6] = 'B', s1 is
Happy Birthday to you, have a great day!
```

Using the Overloaded Operators of Standard Library Class string (Cont.)

- string's at Member Function
  - Class string's at member function throws an exception if its argument is an invalid outof- bounds index.
  - If the index is valid, the at function returns the character at the specified location as a modifiable lvalue or a nonmodifiable lvalue (e.g., a const reference), depending on the context in which the call appears.

Using the Overloaded Operators of Standard Library Class string.

- string's at Member Function
  - Line 83 demonstrates a call to function at with an invalid index causing an out_of_range exception. Here, we show the error message produced by GNU C++.

```
80      // test index out of range with string member function "at"
81      try {
82         std::cout << "Attempt to assign 'd' to s1.at(100) yields:\n";
83         s1.at(100) = 'd'; // ERROR: subscript out of range
84      }
85      catch (const std::out_of_range& ex) {
86         std::cout << std::format("An exception occurred: {}\n", ex.what());
87      }
88   }
```

```
Attempt to assign 'd' to s1.at(100) yields:
An exception occurred: basic_string::at: __n (which is 100) >= this->size()(which is 40)
```

Operator Overloading Fundamentals

- As you saw in the previous example, string's overloaded operators provide a concise notation for manipulating string objects.
- You can use operators with your own user-defined types as well.
- C++ allows you to overload most existing operators by defining overloaded operators.
- Once you define an operator function for a given operator and your custom class, that operator has meaning appropriate for objects of your class.

Operator Overloading Fundamentals (Cont.)

- Operator Overloading Is Not Automatic
  - You must write operator functions that perform the desired operations.
  - You overload an operator by writing a **non-static member function** definition or a **non-member function** definition.
  - **An operator function's name is the keyword operator, followed by the symbol of the operator** you are overloading.
    - For example, the function name **operator+** would overload the addition operator (+).
    - When you overload operators as member functions, they must be non-static: You call them on an object of the class and operate on that object.

Operator Overloading Fundamentals (Cont.)

- Operators That Cannot Be Overloaded
  - The following operators cannot be overloaded:
    - the . (dot) member-selection operator,
    - the .* pointer-to-member operator,
    - the :: scope-resolution operator and
    - the ?: conditional operator
  - The complete list of overloadable operators can be found in https://en.cppreference.com/w/cpp/language/operators

Operator Overloading Fundamentals (Cont.)

- Three Operators That You Do Not Have to Overload
  - For most classes, the **assignment operator (=)** can perform member-wise assignments of the data members.
    - The default = operator assigns each data member from the "source" object (on the right) to the "target" object (on the left).
      - This can be dangerous for classes that have pointer members.
    - So, you'll either explicitly overload the assignment operator or explicitly disallow the compiler from defining the default assignment operator.
    - This is also true for the move assignment operator.
  - **The address (&) operator** returns a pointer to the object.
  - **The comma operator** evaluates the expression to its left, then the expression to its right, and returns the latter expression's value.
    - Though this operator can be overloaded, generally it is not.

Operator Overloading Fundamentals (Cont.)

- Rules and Restrictions on Operator Overloading
  - As you prepare to overload operators for your own classes, there are several rules and restrictions you should keep in mind:
    - An operator's precedence cannot be changed by overloading. Parentheses can be used to change the order of evaluation of overloaded operators in an expression.
    - An operator's grouping (i.e., associativity) cannot be changed by overloading. If an operator normally groups left-to-right, then so do its overloaded versions.
    - An operator's "arity" (the number of operands an operator takes) cannot be changed by overloading. Overloaded unary operators remain unary operators, and overloaded binary operators remain binary operators. The only ternary operator (?:) cannot be overloaded. Operators &, *, + and − each have unary and binary versions that can be overloaded separately.

Operator Overloading Fundamentals (Cont.)

- Rules and Restrictions on Operator Overloading
  - Only existing operators can be overloaded. You cannot create new ones.
  - You cannot overload operators to change how an operator works on only fundamental-type values. For example, you cannot make + subtract two ints.
  - Operator overloading works only with objects of user-defined types or with a mixture of an object of a user-defined type and an object of a fundamental type.
  - Related operators, like + and +=, generally must be overloaded separately.
    - In C++20, if you define == for your class, C++ provides != for you—it simply returns the opposite of ==.

Operator Overloading Fundamentals.

- Rules and Restrictions on Operator Overloading
  - When overloading (), [], -> or =, the operator overloading function must be declared as a class member.
    - You'll see later that this is required when the left operand must be an object of your custom class type.
    - Operator functions for all other overloadable operators may be member or non-member functions.
  - You should overload operators for class types to work as closely as possible to how they work with fundamental types. Avoid excessive

(Downplaying) Dynamic Memory Management with new and delete

- You can allocate and deallocate memory in a program for objects and for arrays of any built-in or user-defined type.
  - This is known as dynamic memory management.
- In Chapter 7, we introduced pointers and showed various old-style techniques you're likely to see in legacy code, then we showed improved Modern C++ techniques.
  - We do the same here.
- For decades, C++ dynamic memory management was performed with operators new and delete.
- The C++ Core Guidelines recommend against using these operators directly.
  - You'll likely see them in legacy C++ code, so we discuss them here.
- Then we will discusses Modern C++ dynamic memory management techniques, which we'll use in the MyArray case study.

Dynamic Memory Management with new and delete (Cont.)

- The Old Way: Operators new and delete
  - You can use the **new operator** to dynamically reserve (that is, allocate) the exact amount of memory required to hold an object or built-in array.
  - **The object or built-in array is created on the free store**—a region of memory assigned to each program for storing dynamically allocated objects.
  - Once memory is allocated, **you can access it via the pointer returned by operator new**.
  - When you no longer need the memory, **you can return it to the free store by using the delete operator** to deallocate (i.e., release) the memory, which can then be reused by future new operations.

Dynamic Memory Management with new and delete (Cont.)

- Obtaining Dynamic Memory with new
  - Consider the following statement:
    Time* timePtr{new Time{}};
  - The new operator allocates memory for a Time object, calls a **default constructor** to initialize it and **returns a Time*—**a pointer of the type specified to the right of the new operator.
    - The preceding statement calls Time's default constructor because we did not supply constructor arguments.
  - If new is unable to find sufficient space in memory for the object, it throws a bad_alloc exception.
    - Chapter 12 shows how to deal with new failures.

Dynamic Memory Management with new and delete (Cont.)

- Releasing Dynamic Memory with delete
  - To destroy a dynamically allocated object and free the space for the object, use **delete**:
    delete timePtr;
  - **This calls the destructor** for the object to which timePtr points, **then deallocates the object's memory**, returning it to the free store.
  - **Not releasing dynamically allocated memory when it's no longer needed can cause memory leaks that eventually lead to a system running out of memory prematurely.**
    - Sometimes the problem might be even worse.
    - If the leaked memory contains objects that manage other resources, those objects' destructors will not be called to release the resources, causing additional leaks.

Dynamic Memory Management with new and delete (Cont.)

- Releasing Dynamic Memory with delete
  - Do not delete memory that was not allocated by new.
    - Doing so results in **undefined behavior.**
    - After you delete dynamically allocated memory, be sure not to delete the same memory again, as this typically causes a program to crash.
    - One way to guard against this is to **immediately set the pointer to nullptr**—deleting such a pointer has no effect.

Dynamic Memory Management with new and delete (Cont.)

- Initializing Dynamically Allocated Objects
  - You can initialize a newly allocated object with constructor arguments, as in
    Time* timePtr{new Time{12, 45, 0}};
    which initializes a new Time object to 12:45:00 PM and assigns its pointer to timePtr.

Dynamic Memory Management with new and delete (Cont.)

- Dynamically Allocating Built-In Arrays with new[]
  - You also can use the new operator to dynamically allocate built-in arrays.
  - The following statement dynamically allocates a 10-element built-in array of ints:
    int* gradesArray{new int[10]{}};
  - This statement aims the int pointer gradesArray at the first element of the dynamically allocated array.
  - The empty braced initializer following new int[10] value initializes the array's elements, which sets fundamental-type elements to 0, bools to false and pointers to nullptr.
    - The braced initializer may also contain a comma-separated list of initializers for the array's elements.

Dynamic Memory Management with new and delete (Cont.)

- Dynamically Allocating Built-In Arrays with new[]
  - Value initializing an object calls its default constructor, if available.
    - The rules become more complicated for objects that do not have default constructors.
    - For more details, see the value-initialization rules at: https://en.cppreference.com/w/cpp/language/value_initialization
  - The size of a built-in array created at compile-time must be specified using an integral constant expression.
  - However, a dynamically allocated array's size can be specified using any non-negative integral expression.

Dynamic Memory Management with new and delete (Cont.)

- Releasing Dynamically Allocated Built-In Arrays with delete[]
  - To deallocate the memory to which gradesArray points, use the statement
    delete[] gradesArray;
  - If the pointer points to a built-in array of objects, this statement **first calls the destructor for each object** in the array, **then deallocates the memory** for the entire array.
  - As with delete, delete[] on a nullptr has no effect.

Dynamic Memory Management with new and delete (Cont.)

- Releasing Dynamically Allocated Built-In Arrays with delete[]
  - Using delete instead of delete[] for an array allocated with new[] results in **undefined behavior.**
    - Some compilers call the destructor only for the first object in the array.
  - To ensure that every object in the array receives a destructor call, **always use delete[] to delete memory allocated by new[]**.
    - We'll show better techniques for managing dynamically allocated memory that enable you to avoid using new and delete.

Dynamic Memory Management with new and delete (Cont.)

- Releasing Dynamically Allocated Built-In Arrays with delete[]
  - If there is only one pointer to a block of dynamically allocated memory and the pointer goes out of scope, or if you assign it nullptr or a different memory address, a memory leak occurs.
  - After deleting dynamically allocated memory, set the pointer's value to nullptr to indicate that it no longer points to memory in the free store.
    - This ensures that your code cannot inadvertently access the previously allocated memory—doing so could cause subtle logic errors.

Dynamic Memory Management with new and delete.

- Range-Based for Does Not Work with Dynamically Allocated Built-In Arrays
  - You might be tempted to use a range-based for statement to iterate over dynamically allocated arrays.
  - Unfortunately, this will not compile.
  - The compiler must know the number of elements at compile time to iterate over an array with range-based for.
  - As a workaround, **you can create a C++20 span object that represents the dynamically allocated array and its number of elements, then iterate over the span object with range-based for**.

Modern C++ Dynamic Memory Management

- A common design pattern is to
  - allocate dynamic memory,
  - assign the address of that memory to a pointer,
  - use the pointer to manipulate the memory and
  - deallocate the memory when it’s no longer needed.
- If an exception occurs after successful memory allocation but before the delete or delete[] statement executes, **a memory leak could occur**.

Modern C++ Dynamic Memory Management (Cont.)

- For this reason, the C++ Core Guidelines recommend that you manage resources like dynamic memory using **RAII—Resource Acquisition Is Initialization**.
- The concept is straightforward. For any resource that must be returned to the system when the program is done using it, the program should
  - create the object as a local variable in a function—t**he object's constructor should acquire the resource while initializing the object**,
  - use that object as necessary in your program, then
  - when the function call terminates, the object goes out of scope—**the object's destructor should release the resource**.

Modern C++ Dynamic Memory Management (Cont.)

- **Smart pointers** use RAII to manage dynamically allocated memory for you.
- The standard library header <memory> defines three smart pointer types:
  - unique_ptr
  - shared_ptr
  - weak_ptr

Modern C++ Dynamic Memory Management (Cont.)

- A unique_ptr maintains a pointer to dynamically allocated memory that can belong to only one unique_ptr at a time.
- When a unique_ptr object goes out of scope, its destructor uses delete or delete[] to deallocate the memory that the unique_ptr manages.
- Here we demonstrate unique_ptr, which will also be used in our MyArray case study.
  - We introduce shared_ptr and weak_ptr in Chapter 20, Other Topics and a Look Toward the Future of C++.

Modern C++ Dynamic Memory Management (Cont.)

- Demonstrating unique_ptr
  - The next program demonstrates a unique_ptr pointing to a dynamically allocated object of class Integer (lines 7–22).

```
1  // main.cpp
2  // Demonstrating unique_ptr.
3  #include <format>
4  #include <iostream>
5  #include <memory>
6
7  class Integer {
8  public:
9     // constructor
10    Integer(int i) : value{i} {
11       std::cout << std::format("Constructor for Integer {}\n", value);
12    }
13
14    // destructor
15    ~Integer() {
16       std::cout << std::format("Destructor for Integer {}\n", value);
17    }
18
19    int getValue() const { return value; } // return Integer value
20 private:
21    int value{0};
22 };
```

Modern C++ Dynamic Memory Management (Cont.)

- Demonstrating unique_ptr
  - Line 29 creates unique_ptr object ptr and initializes it with a pointer to a dynamically allocated Integer object that contains the value 7.
  - To initialize the unique_ptr, line 29 calls the make_unique function template, which uses new to allocate dynamic memory, then returns a unique_ptr to that memory.
  - In this example, make_unique<Integer> returns a unique_ptr<Integer>—line 29 uses the auto keyword to infer ptr's type from its initializer.

```
28      // create a unique_ptr object and "aim" it at a new Integer object
29      auto ptr{std::make_unique<Integer>(7)};
30
31      // use unique_ptr to call an Integer member function
32      std::cout << std::format("Integer value: {}\n\nMain ends\n",
33         ptr->getValue());
34  }
```

```
Creating a unique_ptr that points to
 an Integer
Constructor for Integer 7
Integer value: 7

Main ends
Destructor for Integer 7
```

Modern C++ Dynamic Memory Management (Cont.)

- Demonstrating unique_ptr
  - Line 33 uses the unique_ptr class's overloaded -> operator to invoke getValue on the Integer object that ptr manages.
  - The expression ptr->getValue() also could have been written as
    (*ptr).getValue()
    which uses unique_ptr's overloaded * operator to dereference ptr, then uses the dot (.) operator to invoke getValue on the Integer object.

```
28      // create a unique_ptr object and "aim" it at a new Integer object
29      auto ptr{std::make_unique<Integer>(7)};
30
31      // use unique_ptr to call an Integer member function
32      std::cout << std::format("Integer value: {}\n\nMain ends\n",
33         ptr->getValue());
34  }
```

```
Creating a unique_ptr that points to
 an Integer
Constructor for Integer 7
Integer value: 7

Main ends
Destructor for Integer 7
```

Modern C++ Dynamic Memory Management (Cont.)

- Demonstrating unique_ptr
  - Because ptr is a local variable in main, it's destroyed when main terminates.
  - The unique_ptr destructor deletes the dynamically allocated Integer object, which calls the object's destructor.
  - **The program releases the Integer object's memory, whether program control leaves the block normally via a return statement or reaching the end of the block—or as the result of an exception.**

Modern C++ Dynamic Memory Management (Cont.)

- Demonstrating unique_ptr
  - **Most importantly, using unique_ptr prevents resource leaks.**
    - For example, suppose a function returns a pointer aimed at some dynamically allocated object.
    - Unfortunately, the function caller that receives this pointer might not delete the object, resulting in a memory leak.
    - However, if the function returns a unique_ptr to the object, the object will be deleted automatically when the unique_ptr object's destructor gets called.

Modern C++ Dynamic Memory Management (Cont.)

- unique_ptr Ownership
  - Only one unique_ptr can own a dynamically allocated object, so **assigning one unique_ptr to another transfers ownership** to the target unique_ptr.
    - The same is true when one unique_ptr is passed as an argument to another unique_ptr's constructor.
    - These operations use unique_ptr's move assignment operator and move constructor.
  - **The last unique_ptr object that owns the dynamic memory will delete the memory.**
    - This makes unique_ptr an ideal mechanism for returning ownership of dynamically allocated objects to client code.

Modern C++ Dynamic Memory Management (Cont.)

- unique_ptr to a Built-In Array
  - You can also use a unique_ptr to manage a dynamically allocated built-in array, as we'll do in the MyArray case study.
  - For example, in the statement
    auto ptr{std::make_unique<int[]>(10)};
    make_unique's type argument is specified as int[].
  - So make_unique dynamically allocates a built-in array with the number of elements specified by its argument (10).
    - By default, the int elements are value initialized to 0.
  - The preceding statement uses auto to infer ptr's type (unique_ptr<int[]>) from its initializer.

Modern C++ Dynamic Memory Management.

- unique_ptr to a Built-In Array
  - A unique_ptr that manages an array provides an overloaded subscript operator ([]) to access its elements.
  - For example, the statement
    ptr[2] = 7;
    assigns 7 to the int at ptr[2], and the following statement displays that int:
    std::cout << ptr[2] << "\n";

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Class development is an interesting, creative and intellectually challenging activity—always with the goal of crafting valuable classes.
- When we refer to “arrays” in this case study, we mean the built-in arrays discussed in Chapter 7.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- These pointer-based arrays have many problems, including:
  - C++ does not check whether an array index is out-of-bounds. A program can easily “walk off” either end of an array, likely causing a fatal runtime error if you forget to test for this possibility in your code.
  - Arrays of size n must use index values in the range 0 to n-1. Alternative index ranges are not allowed.
  - You cannot input an entire array with the stream extraction operator (>>) or output one with the stream insertion operator (<<). You must read or write every element.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Two arrays cannot be meaningfully compared with equality or relational operators. Array names are simply pointers to where the arrays begin in memory. Two arrays will always be at different memory locations.
- When you pass an array to a general-purpose function that handles arrays of any size, you must pass the array's size as an additional argument.
  - As you saw in Chapter 7, C++20's spans help solve this problem.
- You cannot assign one array to another with the assignment operator(s).

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- With C++, you can implement more robust array capabilities via classes and operator overloading, as has been done with C++ standard library class templates array and vector. In this section, we'll develop our own custom MyArray class that's preferable to arrays. Internally, class MyArray will use a unique_ptr smart pointer to manage a dynamically allocated built-in array of int values.479

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- We'll create a powerful MyArray class with the following capabilities:
  - MyArrays perform range checking when you access them via the subscript ([]) operator to ensure indices remain within their bounds.
    - Otherwise, the MyArray object will throw a standard library out_of_bounds exception.
  - Entire MyArrays can be input or output with the overloaded stream extraction (>>) and stream insertion (<<) operators without the client-code programmer having to write iteration statements.
  - MyArrays may be compared to one another with the equality operators == and !=. The class could easily be enhanced to include relational operators.
  - MyArrays know their own size, making it easier to pass them to functions.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- MyArray objects may be assigned to one another with the assignment operator.
- MyArrays may be converted to bool (false or true) values to determine whether they are empty or contain elements.
- MyArray provides prefix and postfix increment (++) operators that add 1 to every element.
  - We can easily add prefix and postfix decrement (--) operators.
- MyArray provides an addition assignment operator (+=) that adds a specified value to every element.
  - The class could easily be enhanced to support the -=, *=, /= and %= assignment operators.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Class MyArray will demonstrate the five special member functions and the unique_ptr smart pointer for managing dynamically allocated memory.
- We'll use RAII (Resource Acquisition Is Initialization) throughout this example to manage the dynamically allocated memory resources.
  - The class's constructors will dynamically allocate memory as MyArray objects are initialized.
  - The class's destructor will deallocate the memory when the objects go out of scope, preventing memory leaks.
- Our MyArray class is not meant to replace standard library class templates array and vector, nor is it meant to mimic their capabilities.
- It demonstrates key C++ language and library features that you'll find useful when you build your own classes.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Special Member Functions
  - Every class you define can have five special member functions, each of which we define in class MyArray:
    - a copy constructor,
    - a copy assignment operator,
    - a move constructor,
    - a move assignment operator and
    - a destructor.
  - The copy constructor and copy assignment operator implement the class's copy semantics— that is,
    - how to copy a MyArray when it is passed by value to a function,
    - returned by value from a function or
    - assigned to another MyArray.
  - The move constructor and move assignment operator implement the class's move semantics, which eliminate costly, unnecessary copies of objects that are about to be destroyed.
  - We discuss the details of these special member functions as we encounter the need for them throughout this case study.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Using Class MyArray
  - The next program demonstrates the MyArray class and its rich selection of overloaded operators.
  - The main file tests the various MyArray capabilities.
  - We present the class definition in header file and its member-function definitions in the source code file.
  - For pedagogic purposes, many of class MyArray's member functions, including all its special member functions, display output to show when they're called.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Function getArrayByValue
  - Later in this program, we'll call the getArrayByValue function (lines 10–13) to create a local MyArray object by calling MyArray's constructor that receives an initializer list.
  - Function getArrayByValue returns that local object by value.

```
1   // main.cpp
2   // MyArray class test program.
3   #include <format>
4   #include <iostream>
5   #include <stdexcept>
6   #include <utility> // for std::move
7   #include "MyArray.h"
8
9   // function to return a MyArray by value
10  MyArray getArrayByValue( ) {
11     MyArray localInts{10, 20, 30}; // create three-element MyArray
12     return localInts; // return by value creates an rvalue
13  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Creating MyArray Objects and Displaying Their Size and Contents
  - Lines 16–17 create objects ints1 with seven elements and ints2 with ten elements.
    - Each calls the MyArray constructor that receives the number of elements and initializes the elements to zeros.
  - Lines 20–21 display ints1's size, then output its contents using MyArray's overloaded stream insertion operator (<<).
  - Lines 24–25 do the same for ints2.

```
15  int main() {
16     MyArray ints1(7); // 7-element MyArray; note () rather than {}
17     MyArray ints2(10); // 10-element MyArray; note () rather than {}
18
19     // print ints1 size and contents
20     std::cout << std::format("\nints1 size: {}\ncontents: ", ints1.size())
21        << ints1; // uses overloaded <<
22
23     // print ints2 size and contents
24     std::cout << std::format("\nints2 size: {}\ncontents: ", ints2.size())
25        << ints2; // uses overloaded <<
```

```
MyArray(size_t) constructor
MyArray(size_t) constructor

ints1 size: 7
contents: {0, 0, 0, 0, 0, 0, 0}

ints2 size: 10
contents: {0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Using Parentheses Rather Than Braces to Call the Constructor
  - So far, we've used braced initializers, {}, to pass arguments to constructors.
  - Lines 16–17 use parentheses, (), to call the MyArray constructor that receives a size.
    - We do this because our class—like the standard library's array and vector classes—also supports constructing a MyArray from a braced-initializer list containing the MyArray's element values.
  - When the compiler sees a statement like
    MyArray ints1{7};
    it invokes the constructor that accepts the braced-initializer list of integers, not the single-argument constructor that receives the size.

```
16    MyArray ints1(7); // 7-element MyArray; note ( ) rather than {}
17    MyArray ints2(10); // 10-element MyArray; note ( ) rather than {}
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Using the Overloaded Stream Extraction Operator to Fill a MyArray
  - Next, line 28 prompts the user to enter 17 integers.
  - Line 29 uses the MyArray overloaded stream extraction operator (>>) to read the first seven values into ints1 and the remaining 10 values into ints2 (recall that each MyArray knows its own size).
  - Line 31 displays each MyArray's updated contents using the overloaded stream insertion operator (<<).

```
27    // input and print ints1 and ints2
28    std::cout << "\n\nEnter 17 integers: ";
29    std::cin >> ints1 >> ints2; // uses overloaded >>
30
31    std::cout << "\nints1: " << ints1 << "\nints2: " << ints2;
```

```
Enter 17 integers: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
ints1: {1, 2, 3, 4, 5, 6, 7}
ints2: {8, 9, 10, 11, 12, 13, 14, 15, 16, 17}
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Using the Overloaded Inequality Operator (!=)
  - Line 36 tests MyArray's overloaded inequality operator (!=) by evaluating the condition
    ints1 != ints2
  - The program output shows that the MyArray objects are not equal.
    - Two MyArray objects will be equal if they have the same number of elements and their corresponding element values are identical.
    - As you'll see, we define only MyArray's overloaded == operator.
    - In C++20, the compiler autogenerates != if you provide an == operator for your type. Operator != simply returns the opposite of ==.

```
33    // use overloaded inequality (!=) operator
34    std::cout << "\n\nEvaluating: ints1 != ints2\n";
35
36    if (ints1 != ints2) {
37       std::cout << "ints1 and ints2 are not equal\n\n";
38    }
```

```
Evaluating: ints1 != ints2
ints1 and ints2 are not equal
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Initializing a New MyArray with a Copy of an Existing MyArray
  - Line 41 instantiates the MyArray object ints3 and initializes it with a copy of ints1's data.
    - This invokes the MyArray copy constructor to copy ints1's elements into ints3.
  - A copy constructor is invoked whenever a copy of an object is needed, such as
    - passing an object by value to a function,
    - returning an object by value from a function or
    - initializing an object with a copy of another object of the same class.
  - Lines 44–45 display ints3's size and contents to confirm that ints3's elements were set correctly by the copy constructor.

```
40    // create MyArray ints3 by copying ints1
41    MyArray ints3{ints1}; // invokes copy constructor
42
43    // print ints3 size and contents
44    std::cout << std::format("\nints3 size: {}\ncontents: ", ints3.size())
45       << ints3;
```

```
MyArray copy constructor

ints3 size: 7
contents: {1, 2, 3, 4, 5, 6, 7}
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Initializing a New MyArray with a Copy of an Existing MyArray
  - When a class such as MyArray contains both a copy constructor and a move constructor, the compiler chooses the correct one to use based on the context.
  - In line 41, the compiler chose MyArray's copy constructor because variable names, like ints1, are lvalues.
    - As you'll soon see, a move constructor receives an rvalue reference, which is part of move semantics.
    - An rvalue reference may not refer to an lvalue.
  - The copy constructor can also be invoked by writing line 41 as MyArray ints3 = ints1;
    - In an object's definition, the equal sign does not indicate assignment.
    - **It invokes the single-argument copy constructor, passing as the argument the value to the = symbol's right.**

```
40    // create MyArray ints3 by copying ints1
41    MyArray ints3{ints1}; // invokes copy constructor
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Using the Overloaded Copy Assignment Operator (=)
  - Line 49 assigns ints2 to ints1 to test the overloaded copy assignment operator (=).
    - Built-in arrays **cannot** handle this assignment.
    - An array's name is not a modifiable lvalue, so assigning to an array's name causes a compilation error.
  - Line 51 displays both objects' contents to confirm that they're now identical.
    - MyArray ints1 initially held seven integers, but the overloaded operator resizes the dynamically allocated built-in array to hold a copy of ints2's 10 elements.
    - assignment operator's implementation.

```
47    // use overloaded copy assignment (=) operator
48    std::cout << "\n\nAssigning ints2 to ints1:\n";
49    ints1 = ints2; // note target MyArray is smaller
50
51    std::cout << "\nints1: " << ints1 << "\nints2: " << ints2;
```

```
Assigning ints2 to ints1:
MyArray copy assignment operator
MyArray copy constructor
MyArray destructor

ints1: {8, 9, 10, 11, 12, 13, 14, 15, 16, 17}
ints2: {8, 9, 10, 11, 12, 13, 14, 15, 16, 17}
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Using the Overloaded Copy Assignment Operator (=)
  - As with copy constructors and move constructors, if a class contains both a copy assignment operator and a move assignment operator, **the compiler chooses which one to call based on the arguments**.
    - In this case, ints2 is a variable and thus an lvalue, so the copy assignment operator is called.
    - Note in the output that line 49 also resulted in calls to the MyArray copy constructor and destructor—you'll see why when we present the

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Using the Overloaded Equality Operator (==)
  - Line 56 compares ints1 and ints2 with the overloaded equality operator (==) to confirm they are indeed identical after the assignment in line 49.

```
53    // use overloaded equality (==) operator
54    std::cout << "\n\nEvaluating: ints1 == ints2\n";
55
56    if (ints1 == ints2) {
57       std::cout << "ints1 and ints2 are equal\n\n";
58    }
```

```
Evaluating: ints1 == ints2
ints1 and ints2 are equal
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Using the Overloaded Subscript Operator ([])
  - Line 61 uses the overloaded subscript operator ([]) to refer to ints1[5]—an in-range element of ints1.
    - This indexed (subscripted) name is used to get the value stored in ints1[5].
  - Line 65 uses ints1[5] on an assignment's left side as a modifiable lvalue to assign a new value, 1000, to element 5 of ints1.
    - We'll see that operator[] returns a reference to use as the modifiable lvalue after confirming 5 is a valid index.

```
60  // use overloaded subscript operator to create an rvalue
61  std::cout << std::format("intsl[5] is {}\n\n", intsl[5]);
62
63  // use overloaded subscript operator to create an lvalue
64  std::cout << "Assigning 1000 to intsl[5]\n";
65  intsl[5] = 1000;
66  std::cout << "intsl: " << intsl;
```

```
intsl[5] is 13

Assigning 1000 to intsl[5]
intsl: {8, 9, 10, 11, 12, 1000, 14, 15, 16, 17}
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Using the Overloaded Subscript Operator ([])
  - Line 71 attempts to assign 1000 to ints1[15].
    - This index is outside int1's bounds, so the overloaded operator[] throws an out_of_range exception.
  - Lines 73–75 catch the exception and display its error message by calling the exception's what member function.

```
68    // attempt to use out-of-range subscript
69    try {
70       std::cout << "\n\nAttempt to assign 1000 to ints1[15]\n";
71       ints1[15] = 1000; // ERROR: subscript out of range
72    }
73    catch (const std::out_of_range& ex) {
74       std::cout << std::format("An exception occurred: {}\n", ex.what());
75    }
```

```
Attempt to assign 1000 to ints1[15]
An exception occurred: Index out of range
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Using the Overloaded Subscript Operator ([])
  - The array subscript operator [] can be used with other kinds of objects—for example, to select elements from strings (collections of characters) and maps (collections of key–value pairs).
  - Also, **when overloaded operator[] functions are defined, indices are not required to be integers**.
    - In Chapter 13, we discuss the standard library **map class** that allows indices of other types, such as strings.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Creating MyArray ints4 and Initializing It with the MyArray Returned By Function getArrayByValue
  - Line 80 initializes MyArray ints4 with the result of calling function getArrayByValue (lines 10–13), which creates a local MyArray containing 10, 20 and 30, then returns it by value.
  - Then, lines 82–83 display the new MyArray's size and contents.

```
77    // initialize ints4 with contents of the MyArray returned by
78    // getArrayByValue; print size and contents
79    std::cout << "\nInitialize ints4 with temporary MyArray object\n";
80    MyArray ints4{getArrayByValue()};
81
82    std::cout << std::format("\nints4 size: {}\ncontents: ", ints4.size())
83       << ints4;
```

```
Initialize ints4 with temporary MyArray object
MyArray(initializer_list) constructor

ints4 size: 3
contents: {10, 20, 30}
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Named Return Value Optimization (NRVO)
  - Recall from function getArrayByValue's definition (lines 10–13) that it creates and initializes a local MyArray using the constructor that receives an initializer_list of int values (line 11).
  - This constructor displays MyArray(initializer_list) constructor each time it's called.
  - Next, getArrayByValue returns that local array by value (line 12).

```
 9   // function to return a MyArray by value
10   MyArray getArrayByValue( ) {
11      MyArray localInts{10, 20, 30}; // create three-element MyArray
12      return localInts; // return by value creates an rvalue
13   }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Named Return Value Optimization (NRVO)
  - You might expect that returning an object by value would make a temporary copy of the object for use in the caller.
    - If it did, this would call MyArray's copy constructor to copy the local MyArray.
    - You also might expect the local MyArray object to go out of scope and have its destructor called as getArrayByValue returns to its caller.
  - However, neither the copy constructor nor the destructor displayed lines of output here.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Named Return Value Optimization (NRVO)
  - This is due to a compiler performance optimization called named return value optimization (NRVO).
    - When the compiler sees that a local object is constructed, returned from a function by value, then used to initialize an object in the caller, the compiler instead constructs the object directly in the caller where it will be used.
    - This eliminates the temporary object and extra constructor and destructor calls mentioned above.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Creating MyArray ints5 and Initializing It With the rvalue Returned By Function std::move
  - The copy constructor is called when you initialize one MyArray with another that's represented by an lvalue.
    - A copy constructor copies its argument's contents.
    - This is similar to a text editor's copy-and-paste—after the operation, you have two copies of the data.
  - C++ also supports move semantics, which help the compiler eliminate the overhead of unnecessarily copying objects.
    - A move is similar to a cut-and-paste operation in a text editor—the data gets moved from the cut location to the paste location.
    - **A move constructor moves into a new object the resources of an object that's no longer needed.**
    - Such a constructor **receives an rvalue reference**, which you'll see is declared **with TypeName&&**.
    - Rvalue references may refer only to rvalues. **Typically, these are temporary objects or objects about to be destroyed—called expiring values or xvalues.**

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Creating MyArray ints5 and Initializing It With the rvalue Returned By Function std::move
  - Line 88 uses class MyArray's move constructor to initialize MyArray ints5.
  - Lines 90–91 display the size and contents of the new MyArray.

```
85      // convert ints4 to an rvalue reference with std::move and
86      // use the result to initialize MyArray ints5
87      std::cout << "\n\nInitialize ints5 with result of std::move(ints4)\n";
88      MyArray ints5{std::move(ints4)}; // invokes move constructor
89
90      std::cout << std::format("\nints5 size: {}\ncontents: ", ints5.size())
91         << ints5
92         << std::format("\n\nSize of ints4 is now : {}", ints4.size());
```

```
Initialize ints5 with result of std::move(ints4)
MyArray move constructor

ints5 size: 3
contents: {10, 20, 30}

Size of ints4 is now: 0
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Creating MyArray ints5 and Initializing It With the rvalue Returned By Function std::move
  - The object ints4 is an lvalue—so it cannot be passed directly to MyArray's move constructor.
  - If you no longer need an object's resources, you can convert it from an lvalue to an rvalue reference by passing the object to the standard library function std::move (from header <utility>).
    - This function casts its argument to an rvalue reference, telling the compiler that ints4's contents are no longer needed.
    - So, line 88 forces MyArray's move constructor to be called and ints4's contents are moved into ints5.

```
85    // convert ints4 to an rvalue reference with std::move and
86    // use the result to initialize MyArray ints5
87    std::cout << "\n\nInitialize ints5 with result of std::move(ints4)\n";
88    MyArray ints5{std::move(ints4)}; // invokes move constructor
89
90    std::cout << std::format("\nints5 size: {}\ncontents: ", ints5.size())
91       << ints5
92       << std::format("\n\nSize of ints4 is now : {}", ints4.size());
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Creating MyArray ints5 and Initializing It With the rvalue Returned By Function std::move
  - It's recommended that you use std::move as shown here only if you know the source object passed to std::move will never be used again.
  - Once an object has been moved, two valid operations can be performed with it:
    - destroying it, and
    - using it on the left side of an assignment to give it a new value.
  - In general, you should NOT call member functions on a moved-from object.
    - We do so in line 92 only to prove that the move constructor indeed moved ints4's resources—the output shows that ints4's size is now 0.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Assigning MyArray ints5 to ints4 with the Move Assignment Operator
  - Line 96 uses class MyArray's move assignment operator to move ints5's contents (10, 20 and 30) back into ints4, then
  - lines 98–99 display the size and contents of ints4.

```
94    // move contents of ints5 into ints4
95    std::cout << "\n\nMove ints5 into ints4 via move assignment\n";
96    ints4 = std::move(ints5); // invokes move assignment
97
98    std::cout << std::format("\nints4 size: {}\ncontents: ", ints4.size())
99       << ints4
100      << std::format("\n\nSize of ints5 is now: {}", ints5.size());
101
102   // check if ints5 is empty by contextually converting it to a bool
103   if (ints5) {
104      std::cout << "\n\nints5 contains elements\n";
105   }
106   else {
107      std::cout << "\n\nints5 is empty\n";
108   }
```

```
Move ints5 into ints4 via move assignment
MyArray move assignment operator

ints4 size: 3
contents: {10, 20, 30}

Size of ints5 is now: 0
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Assigning MyArray ints5 to ints4 with the Move Assignment Operator
  - Line 96 explicitly converts the lvalue ints5 to an rvalue reference using std::move.
    - This indicates that ints5 no longer needs its resources, so the compiler can move them into ints4.
    - In this case, the compiler calls MyArray's move assignment operator.

```
94    // move contents of ints5 into ints4
95    std::cout << "\n\nMove ints5 into ints4 via move assignment\n";
96    ints4 = std::move(ints5); // invokes move assignment
97
98    std::cout << std::format("\nints4 size: {}\ncontents: ", ints4.size())
99       << ints4
100      << std::format("\n\nSize of ints5 is now: {}", ints5.size());
101
102   // check if ints5 is empty by contextually converting it to a bool
103   if (ints5) {
104      std::cout << "\n\nints5 contains elements\n";
105   }
106   else {
107      std::cout << "\n\nints5 is empty\n";
108   }
```

```
Move ints5 into ints4 via move assignment
MyArray move assignment operator

ints4 size: 3
contents: {10, 20, 30}

Size of ints5 is now: 0
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Converting MyArray ints5 to a bool to Test Whether It's Empty
  - Many programming languages allow you to use a container-class object like a MyArray as a condition to determine whether the container has elements.
  - For class MyArray, we defined a bool conversion operator that returns true if the MyArray object contains elements (i.e., its size is greater than 0) and false otherwise.
  - **In contexts that require bool values, such as control statement conditions, C++ can invoke an object's bool conversion operator implicitly: This is known as a contextual conversion**.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Converting MyArray ints5 to a bool to Test Whether It's Empty
  - Line 103 uses the MyArray ints5 as a condition, which calls MyArray's bool conversion operator.
  - Since we just moved ints5's resources into ints4, ints5 is now empty, and the operator returns false.

```
102     // check if ints5 is empty by contextually converting it to a bool
103     if (ints5) {
104        std::cout << "\n\nints5 contains elements\n";
105     }
106     else {
107        std::cout << "\n\nints5 is empty\n";
108     }
```

```
ints5 is empty
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Preincrementing Every ints4 Element with the Overloaded ++ Operator
  - Some libraries support **"broadcast" operations** that apply the same operation to every element of a data structure.
  - For example, consider the popular high-performance Python programming language library NumPy.
    - This library's ndarray (n-dimensional array) data structure overloads many arithmetic operators that conveniently perform mathematical operations on every element of an ndarray.
    - In NumPy, the following Python code adds one to every element of the ndarray named numbers—no iteration is required:
      numbers += 1 # Python does not have a ++ operator
  - We've added a similar capability to the MyArray class.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Preincrementing Every ints4 Element with the Overloaded ++ Operator
  - Line 111 displays ints4's current contents, then line 112 uses ++ints4 to preincrement the MyArray, adding one to each element.
    - This expression's result is the updated MyArray.
    - We then use MyArray's overloaded stream insertion operator (<<) to display the contents.

```
110    // add one to every element of ints4 using preincrement
111    std::cout << "\nints4: " << ints4;
112    std::cout << "\npreincrementing ints4: " << ++ints4;
```

```
ints4: {10, 20, 30}
preincrementing ints4: {11, 21, 31}
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Postincrementing Every ints4 Element with the Overloaded ++ Operator
  - Line 115 postincrements the entire MyArray with the expression ints4++.
    - Recall that a postincrement returns its operand's previous value, as confirmed by the program's output.
  - The postincrement operator's implementation, which we'll discuss later, produces the outputs showing that MyArray's copy constructor and destructor were called.

```
114    // add one to every element of ints4 using postincrement
115    std::cout << "\n\npostincrementing ints4: " << ints4++ << "\n";
116    std::cout << "\nints4 now contains: " << ints4;
```

```
postincrementing ints4: MyArray copy constructor
{11, 21, 31}
MyArray destructor
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Adding a Value to Every ints4 Element with the Overloaded += Operator
  - Class MyArray also provides a broadcasting overloaded addition assignment operator (+=) for adding an int value to every MyArray element.
  - Line 119 adds 7 to every ints4 element, then displays its new contents.
  - Note that the non-overloaded versions of ++ and += still work on individual MyArray elements, too—those are simply int values.

```
118     // add a value to every element of ints4 using +=
119     std::cout << "\n\nAdd 7 to every ints4 element: " << (ints4 += 7)
120        << "\n";
121  }
```

```
Add 7 to every ints4 element: {19, 29, 39}
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Destroying the MyArray Objects That Remain
  - When main terminates, the destructors are called for the five MyArray objects created in main, producing the last five lines of the program's output.

```
MyArray destructor
MyArray destructor
MyArray destructor
MyArray destructor
MyArray destructor
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- MyArray Class Definition: Header
  - Lines 53–54 declare MyArray's private data members:
    - m_size stores its number of elements.
    - m_ptr is a unique_ptr that manages a dynamically allocated pointer-based int array containing the MyArray object's elements.
      - When a MyArray goes out of scope, its destructor will call the unique_ptr's destructor, automatically deleting the dynamically allocated memory.
  - Throughout this class's member-function implementations, we use some of C++'s declarative, functional-style programming capabilities discussed in Chapters 6 and 7.
  - We also introduce three additional **standard library algorithms—copy, for_each and equal**.

```
52    private:
53       size_t m_size{0}; // pointer-based array size
54       std::unique_ptr<int[]> m_ptr; // smart pointer to integer array
55    };
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Constructor That Specifies a MyArray's Size
  - Line 16 of the header file
    explicit MyArray(size_t size); // construct a MyArray of size elements
    declares a constructor that specifies the number of MyArray elements.
  - The source code file includes headers used in the example.

```
1   // MyArray.cpp
2   // MyArray class member- and friend-function definitions.
3   #include <algorithm>
4   #include <format>
5   #include <initializer_list>
6   #include <iostream>
7   #include <memory>
8   #include <span>
9   #include <sstream>
10  #include <stdexcept>
11  #include <utility>
12  #include "MyArray.h" // MyArray class definition
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Constructor That Specifies a MyArray's Size
  - The constructor's definition (lines 15–19) performs several tasks:
    - Line 16 initializes the m_size member using the argument size.
    - Line 17 initializes the m_ptr member to a unique_ptr returned by the standard library's make_unique function template.
      - Here, we use it to create a dynamically allocated int array of size elements.
      - The function make_unique value initializes the dynamic memory it allocates, so the int array's elements are set to 0.

```
14  // MyArray constructor to create a MyArray of size elements containing 0
15  MyArray::MyArray(size_t size)
16     : m_size{size},
17     m_ptr{std::make_unique<int[]>(size)} {
18     std::cout << "MyArray(size_t) constructor\n";
19  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- assign a Braced Initializer to a Constructor
  - In Chapter 6, we initialized a std::array object with a braced-initializer list, as in
    std::array<int, 5> n{32, 27, 64, 18, 95};
  - You can use braced initializers for objects of your own classes by providing a constructor with a std::initializer_list parameter, as declared in line 19 of the header:
    explicit MyArray(std::initializer_list<int> list);
    - The std::initializer_list class template is defined in <initializer_list>.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- assign a Braced Initializer to a Constructor
  - With this constructor, we can create MyArray objects that initialize their elements as they're constructed, as in
    MyArray ints{10, 20, 30};
    or
    MyArray ints = {10, 20, 30};
    - Each creates a three-element MyArray containing 10, 20 and 30.
    - **In a class that provides an initializer_list constructor, the class's other single-argument constructors must be called using parentheses rather than braces.**

```
21  // MyArray constructor that accepts an initializer list
22  MyArray::MyArray(std::initializer_list<int> list)
23     : m_size{list.size()}, m_ptr{std::make_unique<int[]>(list.size())} {
24     std::cout << "MyArray(initializer_list) constructor\n";
25
26     // copy list argument's elements into m_ptr's underlying int array
27     // m_ptr.get() returns the int array's starting memory location
28     std::copy(std::begin(list), std::end(list), m_ptr.get());
29  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- assign a Braced Initializer to a Constructor
  - The braced-initialization constructor (lines 22–29) has one initializer_list<int> parameter named list.
  - You can determine list's number of elements by calling its size member function (line 23).

```
21  // MyArray constructor that accepts an initializer list
22  MyArray::MyArray(std::initializer_list<int> list)
23     : m_size{list.size()}, m_ptr{std::make_unique<int[]>(list.size())} {
24     std::cout << "MyArray(initializer_list) constructor\n";
25
26     // copy list argument's elements into m_ptr's underlying int array
27     // m_ptr.get() returns the int array's starting memory location
28     std::copy(std::begin(list), std::end(list), m_ptr.get());
29  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- assign a Braced Initializer to a Constructor
  - To copy each initializer list value into the new MyArray object, line 28 uses the standard library **copy** algorithm (from header <algorithm>) to copy each initializer_list element into the new MyArray.
    - The algorithm copies each element in the range specified by its first two arguments—the beginning and end of the initializer_list into the destination specified by copy's third argument.
    - The unique_ptr's get member function returns the int* that points to the first element of the MyArray's underlying int array.

```
21  // MyArray constructor that accepts an initializer list
22  MyArray::MyArray(std::initializer_list<int> list)
23     : m_size{list.size()}, m_ptr{std::make_unique<int[]>(list.size())} {
24     std::cout << "MyArray(initializer_list) constructor\n";
25
26     // copy list argument's elements into m_ptr's underlying int array
27     // m_ptr.get() returns the int array's starting memory location
28     std::copy(std::begin(list), std::end(list), m_ptr.get());
29  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Copy Constructor and Copy Assignment Operator
  - Chapter 9 introduced the compiler-generated default copy assignment operator and default copy constructor.
    - These performed memberwise copy operations by default.
  - The C++ Core Guidelines recommend designing your classes such that the compiler can autogenerate the copy constructor, copy assignment operator, move constructor, move assignment operator and destructor
    - This is known as the Rule of Zero.
    - You can accomplish this by composing each class's data **using fundamental-type members and objects of classes that do NOT require you to implement custom resource processing**, such as the standard library class vector, which uses RAII to manage resources.

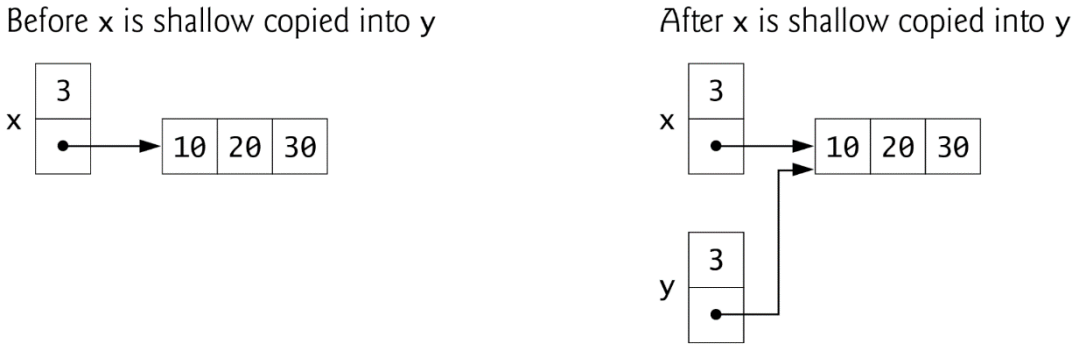
MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- The Rule of Five
  - The default special member functions work well for fundamental-type values like ints and doubles.
  - But what about objects of types that manage their own resources, such as pointers to dynamically allocated memory?
  - Classes that manage their own resources should define the five special member functions.
  - The C++ Core Guidelines state that **if a class requires one special member function, it should define them all**, as we do in this case study.
    - This is known as the Rule of Five.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

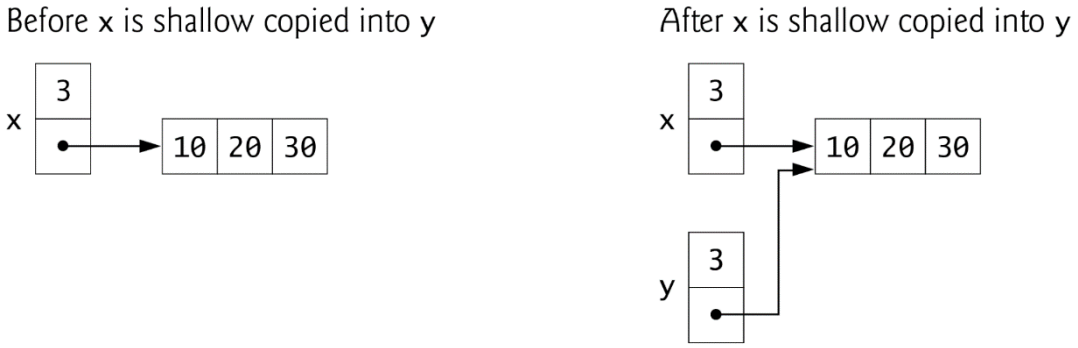
- The Rule of Five
  - Even for classes with the compiler-generated special member functions, some experts recommend declaring them explicitly in the class definition with **= default** (introduced in Chapter 10).
    - This is called the Rule of Five defaults.
  - You also can explicitly remove compiler-generated special member functions to prevent the specified functionality by following their function prototypes with **= delete**.
    - Class unique_ptr actually does this for the copy constructor and copy assignment operator.

MyArray Case Class with Ope



- Shallow Copy
  - The compiler-generated copy constructor and copy assignment operator perform memberwise **shallow copies**.
    - If the member is a pointer to dynamically allocated memory, only the address in the pointer is copied.
    - In the above diagram, consider the object x, which contains its number of elements (3) and a pointer to a dynamically allocated array.
      - For this discussion, let's assume the pointer member is just an int*, not a unique_ptr.

MyArray Case Class with Ope



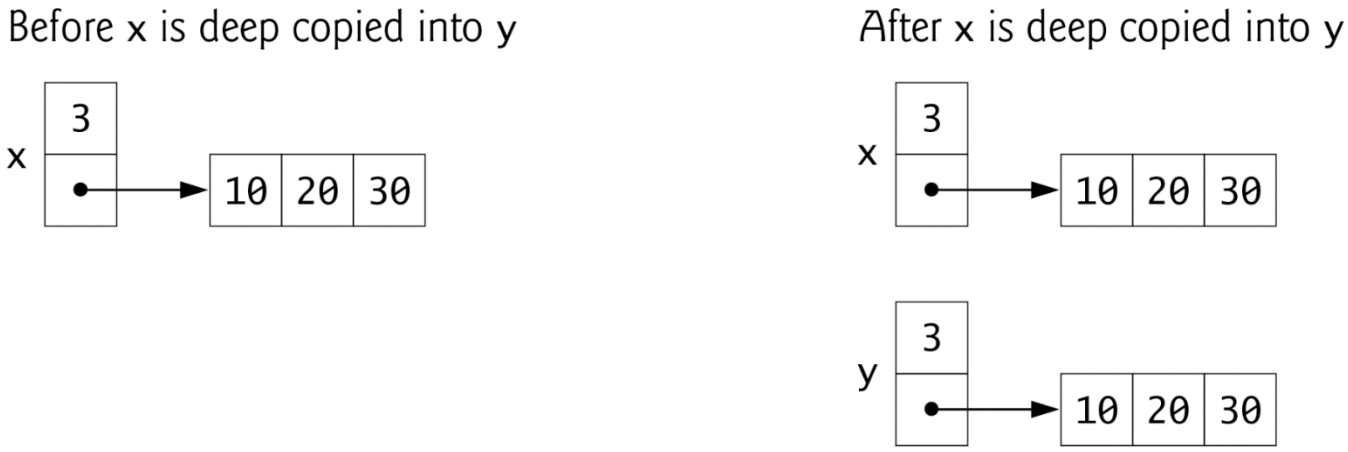
- Shallow Copy
  - Now, assume we'd like to copy x into a new object named y.
  - If the copy constructor simply copied the pointer in x into the target object y's pointer, both would point to the same dynamically allocated memory, as in the right side of the diagram.
    - The first destructor to execute would delete the memory that is shared between these two objects.
    - The other object's pointer would then point to memory that's no longer allocated.
    - This situation is called a dangling pointer. Typically, this would result in a serious runtime error (such as early program termination) if the program were to dereference that pointer—accessing deleted memory is undefined behavior.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Deep Copy
  - In classes that manage their own resources, copying must be done carefully to avoid the pitfalls of shallow copies.
    - Classes that manage their objects' resources should define their own copy constructor and overloaded copy assignment operator to perform deep copies.
    - Not providing a copy constructor and overloaded assignment operator for a class when objects of that class contain pointers to dynamically allocated memory is a potential logic error.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Deep Copy
  - The following diagram shows that after the object x is deep copied into the object y, both objects have their own copies of the dynamically allocated array containing 10, 20 and 30.



MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Implementing the Copy Constructor
  - Line 21 of header
    MyArray(const MyArray& original); // copy constructor
    declares the class's copy constructor (defined lines 32–41).
    - Its argument must be a **const reference** to prevent the constructor from modifying the argument object's data.

```
31  // copy constructor: must receive a reference to a MyArray
32  MyArray::MyArray(const MyArray& original)
33     : m_size{original.size()},
34       m_ptr{std::make_unique<int[]>(original.size())} {
35     std::cout << "MyArray copy constructor\n";
36
37     // copy original's elements into m_ptr's underlying int array
38     const std::span<const int> source{
39        original.m_ptr.get(), original.size()};
40     std::copy(std::begin(source), std::end(source), m_ptr.get());
41  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Implementing the Copy Constructor
  - When the copy constructor is called to initialize a new MyArray by copying an existing one, it performs the following tasks:
    - Line 33 initializes the m_size member using the return value of original's size member function.
    - Line 34 initializes the m_ptr member to a unique_ptr returned by the standard library's make_unique function template, which creates a dynamically allocated int array containing original.size() elements.

```
31  // copy constructor: must receive a reference to a MyArray
32  MyArray::MyArray(const MyArray& original)
33     : m_size{original.size()},
34       m_ptr{std::make_unique<int[]>(original.size())} {
35     std::cout << "MyArray copy constructor\n";
36
37     // copy original's elements into m_ptr's underlying int array
38     const std::span<const int> source{
39        original.m_ptr.get(), original.size()};
40     std::copy(std::begin(source), std::end(source), m_ptr.get());
41  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Implementing the Copy Constructor
  - Line 35 outputs that the copy constructor was called.
  - Lines 38–39 create a span named source representing the argument MyArray's dynamically allocated int array from which we'll copy elements.
    - Recall that a span is a view into a contiguous collection of items, such as an array.
  - Line 40 copies the elements in the range represented by the beginning and end of the source span into the MyArray's underlying int array.

```
31  // copy constructor: must receive a reference to a MyArray
32  MyArray::MyArray(const MyArray& original)
33     : m_size{original.size()},
34       m_ptr{std::make_unique<int[]>(original.size())} {
35     std::cout << "MyArray copy constructor\n";
36
37     // copy original's elements into m_ptr's underlying int array
38     const std::span<const int> source{
39        original.m_ptr.get(), original.size()};
40     std::copy(std::begin(source), std::end(source), m_ptr.get());
41  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Copy Assignment Operator (=)
  - Line 22 of header
    MyArray& operator=(const MyArray& right);
    declares the class's **overloaded copy assignment operator (=)**.
  - This function's definition (lines 44–49) enables one MyArray to be assigned to another, copying the contents from the right operand into the left.
  - When the compiler sees the statement
    ints1 = ints2;
    it invokes member function operator= with the call
    ints1.operator=(ints2)

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Copy Assignment Operator (=)
  - We could implement the overloaded copy assignment operator similarly to the copy constructor.
  - However, there's an elegant way to use the copy constructor to implement the overloaded copy assignment operator—**the copy-and-swap idiom**.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Copy Assignment Operator (=)
  - The idiom operates as follows:
    - First, it copies the argument right into a local MyArray object (temp) using the copy constructor (line 46).
      - If this fails to allocate memory for temp's array, a bad_alloc exception will occur. In this case, the overloaded copy assignment operator will terminate without modifying the object on the assignment's left.
    - Line 47 uses class MyArray's friend function swap (defined in lines 169–172) to exchange the contents of *this (the object on the assignment's left) with temp.
    - Finally, the function returns a reference to the current object (*this in line 48), enabling cascaded MyArray assignments such as x = y = z.

```
43    // copy assignment operator: implemented with copy-and-swap idiom
44    MyArray& MyArray::operator=(const MyArray& right) {
45       std::cout << "MyArray copy assignment operator\n";
46       MyArray temp{right}; // invoke copy constructor
47       swap(*this, temp); // exchange contents of this object and temp
48       return *this;
49    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Copy Assignment Operator (=)
  - When the function returns to its caller, the temp object's destructor is called to release the memory managed by the temp object's unique_ptr.
  - Line 46's copy constructor call and the destructor call when temp goes out of scope are the reason for the two additional lines of output you saw when we demonstrated assigning ints2 to ints1 in line 49 of the main file.

```
43    // copy assignment operator: implemented with copy-and-swap idiom
44    MyArray& MyArray::operator=(const MyArray& right) {
45       std::cout << "MyArray copy assignment operator\n";
46       MyArray temp{right}; // invoke copy constructor
47       swap(*this, temp); // exchange contents of this object and temp
48       return *this;
49    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- friend Function swap
  - Line 13 of header
    friend void swap(MyArray& a, MyArray& b) noexcept;
    declares the swap function **used by the copy assignment operator to implement the copy-and-swap idiom**.
  - This function is declared **noexcept**—exchanging the contents of two existing objects **does NOT allocate new resources, so it should not fail**.

```
168   // swap function used to implement copy-and-swap copy assignment operator
169   void swap(MyArray& a, MyArray& b) noexcept {
170       std::swap(a.m_size, b.m_size); // swap using std::swap
171       a.m_ptr.swap(b.m_ptr); // swap using unique_ptr swap member function
172   }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- friend Function swap
  - The function (lines 169–172) receives two MyArrays.
    - It uses the standard library swap function to exchange the contents of each object's m_size members.
    - It uses the unique_ptr's swap member function to exchange the contents of each object's m_ptr members.

```
168 // swap function used to implement copy-and-swap copy assignment operator
169 void swap(MyArray& a, MyArray& b) noexcept {
170    std::swap(a.m_size, b.m_size); // swap using std::swap
171    a.m_ptr.swap(b.m_ptr); // swap using unique_ptr swap member function
172 }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Move Constructor and Move Assignment Operator
  - Often an object being copied is about to be destroyed, such as a local object returned from a function by value.
  - It's more efficient to move that object's contents into the destination object to eliminate the copying overhead.
    - That's the purpose of the move constructor and move assignment operator.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Move Constructor and Move Assignment Operator
  - In lines 24–25 of header:
    MyArray(MyArray&& original) noexcept; // move constructor
    MyArray& operator=(MyArray&& right) noexcept; // move assignment
    - Each receives an **rvalue reference declared with &&** to distinguish it from an lvalue reference &.
    - Rvalue references help implement move semantics.
      - Rather than copying the argument, the move constructor and move assignment operator each move their argument object's data, leaving the original object in a state that can be destructed properly.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- noexcept Specifier
  - If a function does not throw any exceptions and does not call any functions that throw exceptions, you should explicitly state that the function does not throw exceptions.
  - Simply add noexcept **after the function's signature in both the prototype and the definition**.
    - For a const member function, noexcept must be **placed after const**.
  - If a noexcept function calls another function that throws an exception and the noexcept function does not handle that exception, the program terminates immediately.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- noexcept Specifier
  - The noexcept specification can optionally be followed by parentheses containing a bool expression that evaluates to true or false.
    - noexcept by itself is equivalent to noexcept(true).
  - Following a function's signature with noexcept(false) indicates that the class's designer has thought about whether the function might throw exceptions and **has decided it might**.
    - In such cases, client-code programmers can decide whether to wrap calls to the function in try statements.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Class MyArray's Move Constructor
  - The move constructor (lines 52–56) declares an rvalue reference (&&) parameter, indicating that its MyArray argument must be a temporary object.
  - The member-initializer list moves members m_size and m_ptr from the argument into the object being constructed.

```
51    // move constructor: must receive an rvalue reference to a MyArray
52    MyArray::MyArray(MyArray&& original) noexcept
53       : m_size{std::exchange(original.m_size, 0)},
54       m_ptr{std::move(original.m_ptr)} { // move original.m_ptr into m_ptr
55       std::cout << "MyArray move constructor\n";
56    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Class MyArray's Move Constructor
  - Recall that the only valid operations on a moved-from object are **assigning another object to it or destroying it**.
  - When moving resources from an object, the object should be left in a state that allows it to be properly destructed.
    - Also, it should no longer refer to the resources that were moved to the new object.

```
51    // move constructor: must receive an rvalue reference to a MyArray
52    MyArray::MyArray(MyArray&& original) noexcept
53       : m_size{std::exchange(original.m_size, 0)},
54       m_ptr{std::move(original.m_ptr)} { // move original.m_ptr into m_ptr
55       std::cout << "MyArray move constructor\n";
56    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Class MyArray's Move Constructor
  - To accomplish this for the m_size member, line 53
    - calls the standard library exchange function (header <utility>), which sets its first argument (original.m_size) to its second argument's value (0) and returns its first argument's original value, then
    - initializes the new object's m_size with the value that exchange returns.

```
51    // move constructor: must receive an rvalue reference to a MyArray
52    MyArray::MyArray(MyArray&& original) noexcept
53       : m_size{std::exchange(original.m_size, 0)},
54       m_ptr{std::move(original.m_ptr)} { // move original.m_ptr into m_ptr
55       std::cout << "MyArray move constructor\n";
56    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Class MyArray's Move Constructor
  - When a unique_ptr is move constructed, as in line 54, its **move constructor transfers ownership of the source unique_ptr's dynamic memory to the target unique_ptr and sets the source unique_ptr to nullptr**.
  - If your class manages raw pointers, you'd have to explicitly set the source pointer to nullptr—or use exchange, similar to line 53.

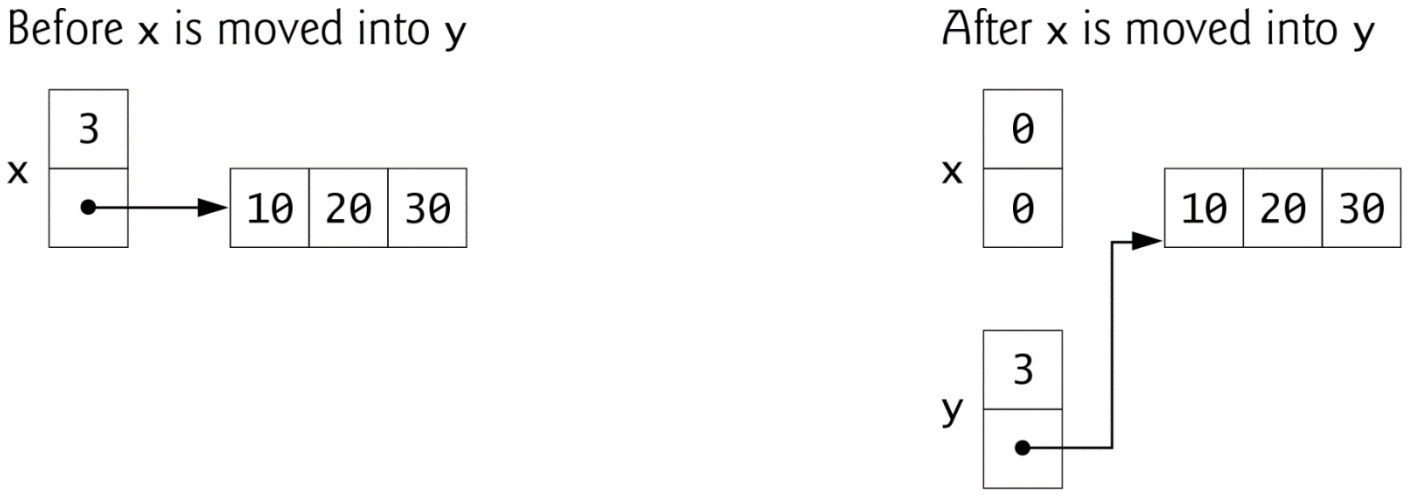
```
51    // move constructor: must receive an rvalue reference to a MyArray
52    MyArray::MyArray(MyArray&& original) noexcept
53       : m_size{std::exchange(original.m_size, 0)},
54       m_ptr{std::move(original.m_ptr)} { // move original.m_ptr into m_ptr
55       std::cout << "MyArray move constructor\n";
56    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Moving Does Not Move Anything
  - Though we said the move constructor "moves the m_size and m_ptr members from the argument object into the object being constructed," **it does not actually move anything**:
    - For fundamental types like size_t, which is simply an unsigned integer, the value is copied from the source object's member to the new object's member.
    - For a raw pointer, the address stored in the source object's pointer is copied to the new object's pointer.
    - For an object, the object's move constructor is called.
      - A unique_ptr's move constructor transfers ownership of the dynamic memory by
        - ✓ copying the dynamically allocated memory's address from the source unique_ptr's underlying raw pointer into the new object's underlying raw pointer, then
        - ✓ assigning nullptr to the source unique_ptr to indicate it no longer manages any data.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Moving Does Not Move Anything
  - The diagram below shows the concept of moving a source object, x, with a raw pointer member into a new object named y.
    - Note that both members of x are 0 after the move—0 in a pointer member represents a null pointer.



MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Class MyArray's Move Assignment Operator (=)
  - The move assignment operator (=) (lines 59–69) defines an rvalue reference (&&) parameter to indicate that its MyArray argument's resources should be moved (not copied).

```
58    // move assignment operator
59    MyArray& MyArray::operator=(MyArray&& right) noexcept {
60       std::cout << "MyArray move assignment operator\n";
61
62       if (this != &right) { // avoid self-assignment
63          // move right's data into this MyArray
64          m_size = std::exchange(right.m_size, 0); // indicate right is empty
65          m_ptr = std::move(right.m_ptr);
66       }
67
68       return *this; // enables x = y = z, for example
69    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Class MyArray's Move Assignment Operator (=)
  - Line 62 tests for self-assignment in which a MyArray object is being assigned to itself.
    - When "this" is equal to the right operand's address, the same object is on both sides of the assignment, so there's no need to move anything.

```
58    // move assignment operator
59    MyArray& MyArray::operator=(MyArray&& right) noexcept {
60       std::cout << "MyArray move assignment operator\n";
61
62       if (this != &right) { // avoid self-assignment
63          // move right's data into this MyArray
64          m_size = std::exchange(right.m_size, 0); // indicate right is empty
65          m_ptr = std::move(right.m_ptr);
66       }
67
68       return *this; // enables x = y = z, for example
69    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Class MyArray's Move Assignment Operator (=)
  - If it is not a self-assignment, lines 64–65
    - move right.m_size into the target MyArray's m_size by calling exchange—this sets right.m_size to 0 and returns its original value, which we use to set the target MyArray's m_size member, and
    - move right.m_ptr into the target MyArray's m_ptr.

```
58  // move assignment operator
59  MyArray& MyArray::operator=(MyArray&& right) noexcept {
60     std::cout << "MyArray move assignment operator\n";
61
62     if (this != &right) { // avoid self-assignment
63        // move right's data into this MyArray
64        m_size = std::exchange(right.m_size, 0); // indicate right is empty
65        m_ptr = std::move(right.m_ptr);
66     }
67
68     return *this; // enables x = y = z, for example
69  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Class MyArray's Move Assignment Operator (=)
  - As in the move constructor, when a unique_ptr is move assigned, ownership of its dynamic memory transfers to the new unique_ptr, and the move assignment operator sets the original unique_ptr to nullptr.
  - Regardless of whether this is a self-assignment, the member function returns the current object (*this), which enables cascaded MyArray assignments such as x = y = z.

```
58    // move assignment operator
59    MyArray& MyArray::operator=(MyArray&& right) noexcept {
60       std::cout << "MyArray move assignment operator\n";
61
62       if (this != &right) { // avoid self-assignment
63          // move right's data into this MyArray
64          m_size = std::exchange(right.m_size, 0); // indicate right is empty
65          m_ptr = std::move(right.m_ptr);
66       }
67
68       return *this; // enables x = y = z, for example
69    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Move Operations Should Be noexcept
  - Move constructors and move assignment operators should never throw exceptions.
    - They do not acquire any new resources—they simply move existing ones.
  - For this reason, the C++ Core Guidelines recommend **declaring move constructors and move assignment operators noexcept**.
    - This is also a requirement to be able to use your class's move capabilities with the standard library's containers like **vector**.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Destructor
  - Line 27 of header
    ~MyArray(); // destructor
    declares the class's destructor (defined in lines 73–75), which is invoked when MyArray object goes out of scope.
  - This will automatically call m_ptr's destructor, which will release the dynamically allocated int array created when the MyArray object was constructed.

```
71    // destructor: This could be compiler-generated. We included it here so
72    // we could output when each MyArray is destroyed.
73    MyArray::~MyArray() {
74       std::cout << "MyArray destructor\n";
75    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Destructor
  - The C++ Core Guidelines indicate that it's a poor design if destructors throw exceptions.
    - For this reason, they recommend declaring destructors noexcept.
    - However, unless your class is derived from a base class with a destructor that's declared noexcept(false), **the compiler implicitly declares the destructor noexcept by default**.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- toString and size Functions
  - Line 29 in header
    size_t size() const noexcept {return m_size;}; // return size
    defines an inline size member function, which returns a MyArray's number of elements.
  - Line 30 in header
    std::string toString() const; // create string representation
    declares a toString member function (defined in lines 78–91), which returns the string representation of a MyArray's contents.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- toString and size Functions
  - Function toString uses an ostringstream to build a string containing the MyArray's element values enclosed in braces ({}) and separating each int from the next by a comma and a space.

```
77  // return a string representation of a MyArray
78  std::string MyArray::toString() const {
79     const std::span<const int> items{m_ptr.get(), m_size};
80     std::ostringstream output;
81     output << "{";
82
83     // insert each item in the dynamic array into the ostringstream
84     for (size_t count{0}; const auto & item : items) {
85        ++count;
86        output << item << (count < m_size ? ", " : "");
87     }
88
89     output << "}";
90     return output.str();
91  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Equality (==) and Inequality (!=) Operators
  - Line 33 of header
    bool operator==(const MyArray& right) const noexcept;
    declares the overloaded equality operator (==).
    - Comparisons should not throw exceptions, so they should be declared noexcept.
  - When the compiler sees an expression like ints1 == ints2, it invokes this overloaded operator with the call ints1.operator==(ints2)

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Equality (==) and Inequality (!=) Operators
  - Member function operator== defined in lines 95–101:
    - Lines 97–98 create the span lhs and rhs representing the dynamically allocated int array in the left-hand-side operand (ints1) and right-hand-side operand (ints2).
    - Lines 99–100 use the standard library algorithm equal (from header <algorithm>) to compare corresponding elements from each span.
      - The first two arguments specify the lhs object's range of elements.
      - The last two specify the rhs object's range of elements.
      - If the lhs and rhs objects have different lengths or if any pair of corresponding elements differ, equal returns false.
      - If every pair of elements is equal, equal returns true.

```
93    // determine if two MyArrays are equal and
94    // return true, otherwise return false
95    bool MyArray::operator==(const MyArray& right) const noexcept {
96       // compare corresponding elements of both MyArrays
97       const std::span<const int> lhs{m_ptr.get(), size()};
98       const std::span<const int> rhs{right.m_ptr.get(), right.size()};
99       return std::equal(std::begin(lhs), std::end(lhs),
100         std::begin(rhs), std::end(rhs));
101   }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Compiler Generates the != Operator
  - As of C++20, the compiler autogenerates a != operator function for you if you provide the == operator for your type.
  - Prior to C++20, if your class required a custom overloaded inequality operator (!=) operator, you'd typically define != to call the == operator function and return the opposite result.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Defining Comparison Operators as Non-Member Functions
  - Each class we've defined so far, including MyArray, declared its single-argument constructor(s) explicit to prevent implicit conversions,
    - as recommended by the C++ Core Guideline "C.164: Avoid Implicit Conversion Operators."
  - You may also come across the C++ Core Guideline "C.86: Make == Symmetric with Respect to Operand Types and noexcept,"
    - which recommends making comparison operators non-member functions **if your class supports implicitly converting objects of other types to your class's type, or vice versa.**
    - This guideline applies to all comparison operators, but we'll discuss == here, as it's the only comparison operator defined in our MyArray class.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Defining Comparison Operators as Non-Member Functions
  - Assume ints is a MyArray and other is an object of class OtherType, which is implicitly convertible to a MyArray.
  - To satisfy Core Guideline C.86, we'd define operator== as a non-member function with the prototype
    bool operator==(const MyArray& left, const MyArray& right) noexcept;
  - This would allow the following mixed-type expressions:
    ints == other
    or
    other == ints

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Defining Comparison Operators as Non-Member Functions
  - There is no operator== definition that provides parameters exactly matching operands of types MyArray and OtherType or OtherType and MyArray.
  - However, **C++ allows one user-defined conversion per expression**.
  - So, if OtherType objects can be implicitly converted to MyArray objects, the compiler will convert other to a MyArray, then call the operator== that receives two MyArrays.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Defining Comparison Operators as Non-Member Functions
  - C++ follows a complex set of overload-resolution rules to determine which function to call for each operator expression.
  - If operator== is a MyArray member function, the left operand must be a MyArray.
  - C++ will not implicitly convert other to a MyArray to call the member function, so the expression
    other == ints
    with an OtherType object on the left causes a compilation error.
    - This might confuse programmers who'd expect this expression to compile, which is why Core Guideline C.86 recommends using a non-member operator== function.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Subscript ([]) Operator
  - Lines 36 and 39 of header
    int& operator[](size_t index);
    cont int& operator[](size_t index) const;
    declare **overloaded subscript operators** (defined in lines 105–112 and 116–123).
  - When the compiler sees an expression like ints1[5], it invokes the appropriate **overloaded operator[] member function** by generating the call
    ints1.operator[](5)

```
103   // overloaded subscript operator for non-const MyArrays;
104   // reference return creates a modifiable lvalue
105   int& MyArray::operator[](size_t index) {
106      // check for index out-of-range error
107      if (index >= m_size) {
108         throw std::out_of_range{"Index out of range"};
109      }
110
111      return m_ptr[index]; // reference return
112   }
```

```
114   // overloaded subscript operator for const MyArrays
115   // const reference return creates a non-modifiable lvalue
116   const int& MyArray::operator[](size_t index) const {
117      // check for subscript out-of-range error
118      if (index >= m_size) {
119         throw std::out_of_range{"Index out of range"};
120      }
121
122      return m_ptr[index]; // returns copy of this element
123   }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Subscript ([]) Operator
  - The compiler calls the **const version of operator[]** (lines 116–123) when the subscript operator is used **on a const MyArray object**.
    - For example, if you pass a MyArray to a function that receives the MyArray as a const MyArray& named z, then the const version of operator[] is required to execute a statement such as std::cout << z[3];
    - Remember that when an object is const, a program can invoke only the object's const member functions.

```
103    // overloaded subscript operator for non-const MyArrays;
104    // reference return creates a modifiable lvalue
105    int& MyArray::operator[](size_t index) {
106       // check for index out-of-range error
107       if (index >= m_size) {
108          throw std::out_of_range{"Index out of range"};
109       }
110
111       return m_ptr[index]; // reference return
112    }
```

```
114    // overloaded subscript operator for const MyArrays
115    // const reference return creates a non-modifiable lvalue
116    const int& MyArray::operator[](size_t index) const {
117       // check for subscript out-of-range error
118       if (index >= m_size) {
119          throw std::out_of_range{"Index out of range"};
120       }
121
122       return m_ptr[index]; // returns copy of this element
123    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Subscript ([]) Operator
  - Each operator[] definition determines whether argument index is in range.
    - If not, it throws an out_of_range exception (header <stdexcept>).
  - If index is in range, the non-const version of operator[] returns the appropriate MyArray element as a reference.
    - This may be used as a modifiable lvalue on an assignment's left side to modify an array element.
    - The const version of operator[] returns a const reference to the appropriate array element.

```
103  // overloaded subscript operator for non-const MyArrays;
104  // reference return creates a modifiable lvalue
105  int& MyArray::operator[](size_t index) {
106     // check for index out-of-range error
107     if (index >= m_size) {
108        throw std::out_of_range{"Index out of range"};
109     }
110
111     return m_ptr[index]; // reference return
112  }
```

```
114  // overloaded subscript operator for const MyArrays
115  // const reference return creates a non-modifiable lvalue
116  const int& MyArray::operator[](size_t index) const {
117     // check for subscript out-of-range error
118     if (index >= m_size) {
119        throw std::out_of_range{"Index out of range"};
120     }
121
122     return m_ptr[index]; // returns copy of this element
123  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Subscript ([]) Operator
  - The subscript operators used in lines 111 and 122 belong to the class unique_ptr.
    - When a unique_ptr manages a dynamically allocated array, unique_ptr's overloaded [] operator enables you to access the array's elements.

```
103    // overloaded subscript operator for non-const MyArrays;
104    // reference return creates a modifiable lvalue
105    int& MyArray::operator[](size_t index) {
106       // check for index out-of-range error
107       if (index >= m_size) {
108          throw std::out_of_range{"Index out of range"};
109       }
110
111       return m_ptr[index]; // reference return
112    }
```

```
114    // overloaded subscript operator for const MyArrays
115    // const reference return creates a non-modifiable lvalue
116    const int& MyArray::operator[](size_t index) const {
117       // check for subscript out-of-range error
118       if (index >= m_size) {
119          throw std::out_of_range{"Index out of range"};
120       }
121
122       return m_ptr[index]; // returns copy of this element
123    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Unary bool Conversion Operator
  - You can define your own conversion operators for converting between types—these are also called **overloaded cast operators**.
  - Line 42 of header
    explicit operator bool() const noexcept {return size() != 0;}
    defines an **inline overloaded operator bool** that
    - converts a MyArray object to true if the MyArray is not empty or false if it is.
  - We declared this operator explicit to prevent the compiler from using it for implicit conversions.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Unary bool Conversion Operator
  - Overloaded conversion operators do not specify a return type to the left of the operator keyword: **The return type is the conversion operator's type**—bool in this case.
  - In main file, line 103 used the MyArray ints5 as an if statement's condition to determine whether it contained elements.
    - In that case, C++ called this operator bool function to perform a **contextual conversion** of the ints5 object to a bool value for use as a condition.
    - You also may call this function explicitly using an expression like: static_cast<bool>(ints5)

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Preincrement Operator
  - You can overload the prefix and postfix increment and decrement operators.
  - The concepts we show for ++ also apply to the −− operators.
    - Later we show how the compiler distinguishes between the prefix and postfix versions.
  - Line 45 of header
    MyArray& operator++();
    declares MyArray's **unary overloaded preincrement operator** (++).
  - When the compiler sees an expression like ++ints4, it invokes MyArray's overloaded preincrement operator (++) function by generating the call
    ints4.operator++()

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Preincrement Operator
  - This invokes the function in lines 126–132, which adds one to each element by
    - creating the span items to represent the dynamically allocated int array, then
    - using the standard library's **for_each algorithm to call a function that performs a task once for each element of the span**.
    - Like the copy algorithm, for_each's first two arguments represent the range of elements to process.

```
125   // preincrement every element, then return updated MyArray
126   MyArray& MyArray::operator++() {
127      // use a span and for_each to increment every element
128      const std::span<int> items{m_ptr.get(), m_size};
129      std::for_each(std::begin(items), std::end(items),
130         [](auto& item) {++item; });
131      return *this;
132   }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Preincrement Operator
  - The third argument is a function that receives one argument and performs a task with it.
    - In this case, we specify a **lambda expression** that is called once for each element in the range.
    - As **for_each iterates internally through the span's elements**, it passes the current element as the lambda's argument (item).
      - The lambda then performs a task using that value.
    - This lambda's argument is a non-const reference (auto&), so the expression ++item in the lambda's body modifies the original element in the MyArray.

```
125   // preincrement every element, then return updated MyArray
126   MyArray& MyArray::operator++() {
127      // use a span and for_each to increment every element
128      const std::span<int> items{m_ptr.get(), m_size};
129      std::for_each(std::begin(items), std::end(items),
130         [](auto& item) {++item; });
131      return *this;
132   }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Preincrement Operator
  - The operator returns a reference to the MyArray object.
    - This enables a preincremented MyArray object to be used as an lvalue, which is how the built-in prefix increment operator works for fundamental types.

```
125   // preincrement every element, then return updated MyArray
126   MyArray& MyArray::operator++() {
127       // use a span and for_each to increment every element
128       const std::span<int> items{m_ptr.get(), m_size};
129       std::for_each(std::begin(items), std::end(items),
130           [](auto& item) {++item; });
131       return *this;
132   }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Postincrement Operator
  - Overloading the postfix increment operator presents a challenge.
    - The compiler must be able to distinguish between the signatures of the overloaded prefix and postfix increment operator functions.
  - By convention, when the compiler sees a postincrement expression like ints4++, it generates the member-function call ints4.operator++(0)
    - The argument 0 is strictly a dummy value that the compiler uses to distinguish between the prefix and postfix increment operator functions.
    - The same syntax differentiates the prefix and postfix decrement operator functions.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Postincrement Operator
  - Line 48 of header
    MyArray operator++(int);
    declares MyArray's **unary overloaded postincrement operator** (++) with the int parameter that receives the dummy value 0.
    - The parameter is not used, so it's declared without a parameter name.

```
134  // postincrement every element, and return copy of original MyArray
135  MyArray MyArray::operator++(int) {
136     MyArray temp(*this);
137     ++(*this); // call preincrement operator++ to do the incrementing
138     return temp; // return the temporary copy made before incrementing
139  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Postincrement Operator
  - To emulate the effect of the postincrement, we must return an unincremented copy of the MyArray object. So, the function's definition (lines 135–139)
    - uses the MyArray copy constructor to make a local copy of the original MyArray,
    - calls the preincrement operator to add one to every element of the MyArray, then
    - returns the **unincremented local copy of the MyArray by value**—this is another case in which compilers can use the **named return value optimization (NRVO)**.

```
134  // postincrement every element, and return copy of original MyArray
135  MyArray MyArray::operator++(int) {
136     MyArray temp(*this);
137     ++(*this); // call preincrement operator++ to do the incrementing
138     return temp; // return the temporary copy made before incrementing
139  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Postincrement Operator
  - The extra local object created by the postfix increment (or decrement) operator can result in a performance problem, especially if the operator is used in a loop.
  - For this reason, **you should prefer the prefix increment and decrement operators**.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Addition Assignment Operator (+=)
  - Line 51 of header
    MyArray& operator+=(int value);
    declares MyArray's overloaded addition assignment operator (+=), which adds a value to every element of a MyArray, then returns a reference to the modified object to enable cascaded calls.

```
141   // add value to every element, then return updated MyArray
142   MyArray& MyArray::operator+=(int value) {
143      // use a span and for_each to increment every element
144      const std::span<int> items{m_ptr.get(), m_size};
145      std::for_each(std::begin(items), std::end(items),
146         [value](auto& item) {item += value; });
147      return *this;
148   }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Addition Assignment Operator (+=)
  - Like the preincrement operator, we use a span and the standard library function for_each to process every element in the MyArray.
    - In this case, the lambda we pass as for_each's last argument (line 146) uses the operator+= function's value parameter in its body.
    - **The lambda introducer [value] specifies that the compiler should capture value for use in the lambda's body.**

```
141    // add value to every element, then return updated MyArray
142    MyArray& MyArray::operator+=(int value) {
143       // use a span and for_each to increment every element
144       const std::span<int> items{m_ptr.get(), m_size};
145       std::for_each(std::begin(items), std::end(items),
146          [value](auto& item) {item += value; });
147       return *this;
148    }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Binary Stream Extraction (>>) and Stream Insertion (<<) Operators
  - You can input and output fundamental-type data using the stream extraction operator (>>) and the stream insertion operator (<<).
  - The C++ standard library overloads these operators for each fundamental type, including pointers and char* strings.
  - You also can overload these to perform input and output for custom types.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Overloading the Binary Stream Extraction (>>) and Stream Insertion (<<) Operators
  - Lines 10 and 58 of header
    friend std::istream& operator>>(std::istream& in, MyArray& a);
    std::ostream& operator<<(std::ostream& out, const MyArray& a);
    declare the **non-member overloaded stream-extraction operator (>>) and overloaded stream-insertion operator (<<)**.
  - We declared operator>> in the class as a friend because it will access a MyArray's private data directly for performance.
  - The operator<< is NOT declared as a friend.
    - As you'll see, it calls MyArray's toString member function to get a MyArray's string representation, then outputs it.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Implementing the Stream Extraction Operator
  - The overloaded stream-extraction operator (>>) (lines 152–160) takes an istream reference and a MyArray reference as arguments.
  - It returns the istream reference argument to enable cascaded inputs, like
    std::cin >> ints1 >> ints2;
  - When the compiler sees an expression like cin >> ints1, it invokes non-member function operator>> with the call operator>>(std::cin, ints1)
    - We'll say why this needs to be a non-member function.
    - When this call completes, it returns a reference to cin, which would then be used in the cin statement above to input values into ints2.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Implementing the Stream Extraction Operator
  - The function creates a span (line 153) representing MyArray's dynamically allocated int array.
  - Then lines 155–157 iterate through the span's elements, reading one value at a time from the input stream and placing the value in the corresponding element of the dynamically allocated int array.

```
150  // overloaded input operator for class MyArray;
151  // inputs values for entire MyArray
152  std::istream& operator>>(std::istream& in, MyArray& a) {
153     std::span<int> items{a.m_ptr.get(), a.m_size};
154
155     for (auto& item : items) {
156        in >> item;
157     }
158
159     return in; // enables cin >> x >> y;
160  }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Implementing the Stream Insertion Operator
  - The overloaded stream insertion function (<<) (lines 163–166) receives an ostream reference and a const MyArray reference as arguments and returns an ostream reference.
    - The function calls MyArray's toString member function, then outputs the resulting string.
  - The function returns the ostream reference argument to enable cascaded output statements, like std::cout << ints1 << ints2;
  - When the compiler sees an expression like std::cout << ints1, it invokes non-member function operator<< with the call operator<<(std::cout, ints1)

```
162   // overloaded output operator for class MyArray
163   std::ostream& operator<<(std::ostream& out, const MyArray& a) {
164      out << a.toString();
165      return out; // enables std::cout << x << y;
166   }
```

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Why operator>> and operator<< Must Be Non-member Functions
  - The operator>> and operator<< functions are defined as **non-member functions**, so we can **specify their operands' order** in each function's parameter list.
    - In a binary overloaded operator implemented as a non-member function, the first parameter is the left operand, and the second is the right operand.
  - For the operators >> and <<, the MyArray object should be each operator's right operand, so you can use them the way C++ programmers expect, as in the statements
    std::cin >> ints4;
    std::cout << ints4;

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Why operator>> and operator<< Must Be Non-member Functions
  - If we defined these functions as MyArray member functions, programmers would have to write the following awkward statements to input or output MyArray objects:
    ints4 >> std::cin;
    ints4 << std::cout;
    - Such statements would be confusing and, in some cases, would lead to compilation errors.
    - Programmers are familiar with cin and cout always appearing to the left of these operators.
  - Overloaded binary operators may be member functions only for the class of the operator's left operand.
    - For operator>> and operator<< to be member functions, we'd have to modify the standard library classes istream and ostream, which is not allowed.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading (Cont.)

- Choosing Member vs. Non-Member Functions
  - Overloaded operator functions, and functions in general, can be
    - member functions with direct access to the class's internal implementation details,
    - friend functions with direct access to the class's internal implementation details or
    - non-member, non-friend functions—often called free functions—that interact with objects of the class through its public interface.

MyArray Case Study: Crafting a Valuable Class with Operator Overloading.

- Choosing Member vs. Non-Member Functions
  - The C++ Core Guidelines say a function should be a member only if it needs direct access to the class's internal implementation details, such as its private data.
  - Another reason to use non-member functions is to define commutative operators.
  - Consider a class HugeInt for arbitrary-sized integers. With a HugeInt named bigInt, we might write expressions like bigInt + 77 + bigInt
    - Each would sum an int value and a HugeInt.
    - Like the built-in + operator for fundamental types, each would produce a temporary HugeInt containing the sum.
  - To support these expressions, you define two versions of operator+, typically as non-member friend functions:
    - friend HugeInt operator+(const HugeInt& left, int right);
    - friend HugeInt operator+(int left, const HugeInt& right);
    - To avoid code duplication, the second function typically would call the first.

C++20 Three-Way Comparison Operator (<=>)

- You'll often compare objects of your custom class types.
  - For example, to sort objects into ascending or descending order using the standard library's sort function, the objects must be comparable.
- To support comparisons, you can overload the equality (== and !=) and relational (<, <=, > and >=) operators for your classes.
- A common practice is to
  - define the < and == operator functions, then
  - define !=, <=, > and >= in terms of < and ==.

C++20 Three-Way Comparison Operator (<=>) (Cont.)

- For instance, for a class Time that represents the time of day, its <= operator could be implemented as an inline member function in terms of the < operator:
  bool operator<=(const Time& right) const { return !(right < *this); }
  - This leads to lots of "boilerplate" code in which only the argument type differs in the definitions of the overloaded !=, <=, > and >= operators among classes.
- For most types, the compiler can handle the comparison operators for you via the compiler-generated default implementation of C++20's **three-way comparison operator** (<=>) which is also referred to as **the spaceship operator** and requires the header <compare>.

C++20 Three-Way Comparison Operator (<=>) (Cont.)

- The following program demonstrates <=> for a class Time (lines 8–23), which contains
  - a constructor (lines 10–11) to initialize its private data members (lines 20–22),
  - a toString function (lines 13–15) to create a Time's string representation and
  - a defaulted definition of the overloaded <=> operator (line 18).

```
1   // main.cpp
2   // C++20 three-way comparison (spaceship) operator.
3   #include <compare>
4   #include <format>
5   #include <iostream>
6   #include <string>
7
8   class Time {
9   public:
10     Time(int hr, int min, int sec) noexcept
11        : m_hr{hr}, m_min{min}, m_sec{sec} {}
12
13     std::string toString() const {
14        return std::format("hr={}, min={}, sec={}", m_hr, m_min, m_sec);
15     }
16
17     // <=> operator automatically supports equality/relational operators
18     auto operator<=>(const Time& t) const noexcept = default;
19  private:
20     int m_hr{0};
21     int m_min{0};
22     int m_sec{0};
23  };
```

C++20 Three-Way Comparison Operator (<=>) (Cont.)

- The default compiler-generated operator<=> works for any class **containing data members that all support the equality and relational operators**.
- Also, for classes containing **built-in arrays as data members**, **the compiler applies the overloaded operator<=> element-by-element** as it compares two objects of the arrays' element type.

C++20 Three-Way Comparison Operator (<=>) (Cont.)

- In main, lines 26–28 create three Time objects that we'll use in various comparisons, then display their string representations.
  - Time objects t1 and t2 both represent 12:15:30 PM, and t3 represents 6:30:00 AM.

```
25  int main() {
26     const Time t1(12, 15, 30);
27     const Time t2(12, 15, 30);
28     const Time t3(6, 30, 0);
29
30     std::cout << std::format("t1: {}\nt2: {}\nt3: {}\n\n",
31        t1.toString(), t2.toString(), t3.toString());
```

```
t1: hr=12, min=15, sec=30
t2: hr=12, min=15, sec=30
t3: hr=6, min=30, sec=0
```

C++20 Three-Way Comparison Operator (<=>) (Cont.)

- With <=>, the Compiler Supports All the Comparison Operators
  - When you let the compiler generate the overloaded three-way comparison operator (<=>) for you, your class supports all the relational and equality operators, which we demonstrate in lines 34–46.
  - In each case, the compiler rewrites an expression like
    t1 == t2
    into an expression that uses the <=> operator, as in
    (t1 <=> t2) == 0
  - The expression t1 <=> t2 evaluates to
    - 0 if the objects are equal,
    - a negative value if t1 is less than t2, and
    - A positive value if t1 is greater than t2.
  - As you can see, lines 34–46 compiled and produced correct outputs, even though class Time did not define any equality or relational operators.

C++20 Three-Way Comparison Operator (<=>) (Cont.)

- With <=>, the Compiler Supports All the Comparison Operators
  - As you can see, lines 34–46 compiled and produced correct outputs, even though class Time did not define any equality or relational operators.

```
33      // using the equality and relational operators
34      std::cout << std::format("t1 == t2: {}\n", t1 == t2);
35      std::cout << std::format("t1 != t2: {}\n", t1 != t2);
36      std::cout << std::format("t1 < t2: {}\n", t1 < t2);
37      std::cout << std::format("t1 <= t2: {}\n", t1 <= t2);
38      std::cout << std::format("t1 > t2: {}\n", t1 > t2);
39      std::cout << std::format("t1 >= t2: {}\n\n", t1 >= t2);
40
41      std::cout << std::format("t1 == t3: {}\n", t1 == t3);
42      std::cout << std::format("t1 != t3: {}\n", t1 != t3);
43      std::cout << std::format("t1 < t3: {}\n", t1 < t3);
44      std::cout << std::format("t1 <= t3: {}\n", t1 <= t3);
45      std::cout << std::format("t1 > t3: {}\n", t1 > t3);
46      std::cout << std::format("t1 >= t3: {}\n\n", t1 >= t3);
```

```
t1 == t2: true
t1 != t2: false
t1 < t2: false
t1 <= t2: true
t1 > t2: false
t1 >= t2: true

t1 == t3: false
t1 != t3: true
t1 < t3: false
t1 <= t3: false
t1 > t3: true
t1 >= t3: true
```

C++20 Three-Way Comparison Operator (<=>).

- Using <=> Explicitly
  - You also can use <=> in expressions.
  - An <=> expression's result is not convertible to bool, so you must compare it to 0 to use <=> in a condition, as shown in lines 49, 53 and 57.

```
48      // using <=> to perform comparisons
49      if ((t1 <=> t2) == 0) {
50         std::cout << "t1 is equal to t2\n";
51      }
52
53      if ((t1 <=> t3) > 0) {
54         std::cout << "t1 is greater than t3\n";
55      }
56
57      if ((t3 <=> t1) < 0) {
58         std::cout << "t3 is less than t1\n";
59      }
60   }
```

```
t1 is equal to t2
t1 is greater than t3
t3 is less than t1
```

Converting Between Types

- Most programs process information of many types.
- Sometimes all the operations “stay within a type.”
  - For example, adding an int to an int produces an int.
- It’s often necessary, however, to convert data of one type to data of another type.
  - This can happen in assignments, calculations, passing values to functions and returning values from functions.
- The compiler knows how to perform certain conversions among fundamental types.
  - You can use cast operators to force conversions among fundamental types.

Converting Between Types

- But what about user-defined types? The compiler does NOT know how to convert among user-defined types or between user-defined types and fundamental types.
  - You must specify how to do this.
- Such conversions can be performed with conversion constructors.
  - These constructors are called with one argument (we refer to these as single-argument constructors).
  - Such constructors can turn objects of other types (including fundamental types) into objects of a particular class.

Converting Between Types (Cont.)

- Conversion Operators
  - A **conversion operator** (also called a **cast operator**) also can convert an object of a class to another type.
  - Such a conversion operator **must be a non-static member function**.
  - In the MyArray case study, we implemented an overloaded conversion operator that converted a MyArray to a bool value to determine whether the MyArray contained elements.

Converting Between Types (Cont.)

- Implicit Calls to Cast Operators and Conversion Constructors
  - A feature of cast operators and conversion constructors is that the compiler can call them implicitly to create objects. For example, you saw that when an object of our class MyArray appears in a program where a bool is expected, such as
    if (ints1) { // if expects a condition
    ...
    }
    the compiler can call the overloaded cast-operator function operator bool to convert the object into a bool and use the resulting bool in the expression.

Converting Between Types.

- Implicit Calls to Cast Operators and Conversion Constructors
  - When a conversion constructor or conversion operator is used to perform an implicit conversion, **C++ can apply only one implicit constructor or operator function call** (i.e., a single user-defined conversion) **per expression to try to match the needs of that expression**.
  - The compiler will NOT satisfy an expression's needs by performing a series of implicit, user-defined conversions.

explicit Constructors and Conversion Operators

- Recall that we've been declaring as explicit every constructor that can be called with one argument, including multi-parameter constructors for which we specify default arguments.
- Except for copy and move constructors, any constructor that can be called with a single argument and is not declared explicit can be used by the compiler to perform an implicit conversion.
- The constructor's argument is converted to an object of the class in which the constructor is defined.
- The conversion is automatic—a cast is not required.

explicit Constructors and Conversion Operators

- In some situations, implicit conversions are undesirable or error-prone.
  - For example, our MyArray class defines a constructor that takes a single size_t argument.
  - The intent of this constructor is to create a MyArray object containing a specified number of elements.
  - However, if this constructor were not declared explicit, it could be misused by the compiler to perform an implicit conversion.
- Unfortunately, the compiler might use implicit conversions in cases that you do not expect, resulting in execution-time logic errors or ambiguous expressions that generate compilation errors.

explicit Constructors and Conversion Operators (Cont.)

- Accidentally Using a Single-Argument Constructor as a Conversion Constructor
  - The program uses MyArray class to demonstrate an improper implicit conversion.
    - To allow this implicit conversion, we removed the explicit keyword from line 16 in MyArray.h.

```
1  // main.cpp
2  // Single-argument constructors and implicit conversions.
3  #include <iostream>
4  #include "MyArray.h"
5  using namespace std;
6
7  void outputArray(const MyArray&); // prototype
8
9  int main() {
10    MyArray intsl(7); // 7-element MyArray
11    outputArray(intsl); // output MyArray intsl
12    outputArray(3); // convert 3 to a MyArray and output the contents
13 }
14
15 // print MyArray contents
16 void outputArray(const MyArray& arrayToOutput) {
17    std::cout << "The MyArray received has " << arrayToOutput.size()
18       << " elements. The contents are: " << arrayToOutput << "\n";
19 }
```

```
MyArray(size_t) constructor
The MyArray received has 7 elements. The
contents are: {0, 0, 0, 0, 0, 0, 0}
MyArray(size_t) constructor
The MyArray received has 3 elements. The
contents are: {0, 0, 0}
MyArray destructor
MyArray destructor
```

explicit Constructors and Conversion Operators (Cont.)

- Accidentally Using a Single-Argument Constructor as a Conversion Constructor
  - Line 10 in main instantiates MyArray object ints1 and calls the single-argument constructor with the value 7 to specify ints1's number of elements.
  - Recall that the MyArray constructor that receives a size_t argument initializes all the MyArray elements to 0.
  - Line 11 calls function outputArray (defined in lines 16–19), which receives as its argument a const MyArray&.
    - The function outputs its argument's number of elements and contents.
    - In this case, the MyArray's size is 7, so outputArray displays seven 0s.

explicit Constructors and Conversion Operators (Cont.)

- Accidentally Using a Single-Argument Constructor as a Conversion Constructor
  - Line 12 calls outputArray with the value 3 as an argument.
    - This program does not contain an outputArray function that takes an int argument.
    - So, the compiler determines whether the argument 3 can be converted to a MyArray object.
    - Because class MyArray provides a constructor with one size_t argument (which can receive an int) and that constructor is not declared explicit, the compiler assumes the constructor is a conversion constructor and uses it to convert the argument 3 into a temporary MyArray object containing three elements.
    - Then, the compiler passes this temporary MyArray object to function outputArray, which displays the temporary MyArray's size and contents.
      - Thus, even though we do not explicitly provide an outputArray function that receives an int, the compiler can compile line 12.
      - The output shows the contents of the three-element MyArray containing 0s.

explicit Constructors and Conversion Operators (Cont.)

- Preventing Implicit Conversions with Single-Argument Constructors
  - The reason we've declared every single-argument constructor explicit is to suppress implicit conversions via conversion constructors when such conversions should not be allowed.
    - A constructor that's declared explicit cannot be used in an implicit conversion.
  - Our next program uses class MyArray, which included the keyword explicit in the declaration of its single-argument constructor that receives a size_t:
    explicit MyArray(size_t size);

explicit Constructors and Conversion Operators (Cont.)

- Preventing Implicit Conversions with Single-Argument Constructors
  - Here we present a slightly modified version of the previous program.

```
1   // main.cpp
2   // Demonstrating an explicit constructor.
3   #include <iostream>
4   #include "MyArray.h"
5   using namespace std;
6
7   void outputArray(const MyArray&); // prototype
8
9   int main() {
10     MyArray intsl{7}; // 7-element MyArray
11     outputArray(intsl); // output MyArray intsl
12     outputArray(3); // convert 3 to a MyArray and output its contents
13     outputArray(MyArray(3)); // explicit single-argument constructor call
14  }
15
16  // print MyArray contents
17  void outputArray(const MyArray& arrayToOutput) {
18     std::cout << "The MyArray received has " << arrayToOutput.size()
19        << " elements. The contents are: " << arrayToOutput << "\n";
20  }
```

explicit Constructors and Conversion Operators (Cont.)

- Preventing Implicit Conversions with Single-Argument Constructors
  - When this program is compiled, the compiler produces an error message, such as
    error: invalid initialization of reference of type ‘const MyArray&’from expression of type ‘int’
    on g++, indicating that the integer value passed to outputArray in line 12 cannot be converted to a const MyArray&.

```
 9  int main() {
10     MyArray ints1{7}; // 7-element MyArray
11     outputArray(ints1); // output MyArray ints1
12     outputArray(3); // convert 3 to a MyArray and output its contents
13     outputArray(MyArray(3)); // explicit single-argument constructor call
14  }
15
16  // print MyArray contents
17  void outputArray(const MyArray& arrayToOutput) {
18     std::cout << "The MyArray received has " << arrayToOutput.size()
19        << " elements. The contents are: " << arrayToOutput << "\n";
20  }
```

explicit Constructors and Conversion Operators (Cont.)

- Preventing Implicit Conversions with Single-Argument Constructors
  - Line 13 demonstrates how the explicit constructor can be used to create a temporary MyArray of 3 elements and pass it to outputArray.

```
1  // main.cpp
2  // Demonstrating an explicit constructor.
3  #include <iostream>
4  #include "MyArray.h"
5  using namespace std;
6
7  void outputArray(const MyArray&); // prototype
8
9  int main() {
10    MyArray intsl{7}; // 7-element MyArray
11    outputArray(intsl); // output MyArray intsl
12    outputArray(3); // convert 3 to a MyArray and output its contents
13    outputArray(MyArray(3)); // explicit single-argument constructor call
14 }
15
16 // print MyArray contents
17 void outputArray(const MyArray& arrayToOutput) {
18    std::cout << "The MyArray received has " << arrayToOutput.size()
19       << " elements. The contents are: " << arrayToOutput << "\n";
20 }
```

explicit Constructors and Conversion Operators (Cont.)

- Preventing Implicit Conversions with Single-Argument Constructors
  - You should always use the explicit keyword on single-argument constructors,
  - unless they're intended to be used implicitly as conversion constructors.
    - In this case, you should declare the single-argument constructor **explicit(false)**.
    - This documents for your class's users that you wish to allow implicit conversions to be performed with that constructor.

explicit Constructors and Conversion Operators.

- explicit Conversion Operators
  - Just as you can declare single-argument constructors explicit, you can declare conversion operators explicit—or in C++20, explicit(true)—to prevent the compiler from using them to perform implicit conversions.
  - For example, in class MyArray, the prototype
    explicit operator bool() const noexcept;
    declares the bool cast operator explicit, so you'd generally have to invoke it explicitly with static_cast, as in static_cast<bool>(myArrayObject)
  - The static_cast operator explicitly converts its argument to the type in angle brackets.
  - **As we showed in the MyArray case study, C++ can still perform contextual conversions using an explicit bool conversion operator to convert an object to a bool value in a condition.**

Overloading the Function Call Operator ()

- Overloading the function-call operator () is powerful because functions can take an arbitrary number of comma-separated parameters.
- We demonstrate the overloaded function-call operator in a natural context in Chapter 14.